

# Lecture Note 3: Introduction to ESP32

CSS452 Internet of Things

Dr. Seksan Laitrakun

School of ICT, SIIT, Thammasat University

1. ESP32 Specification
2. Arduino IDE Setup and Uploading
3. Basic Features of ESP32
4. WiFi Connection

- Random Nerd Tutorials: <https://randomnerdtutorials.com/projects-esp32/>
- Last Minute Engineers: <https://lastminuteengineers.com/esp32-arduino-ide-tutorial/>
- Thai Easy Elect (Thai Language): <https://blog.thaieasyelec.com/espino32-article-table-of-contents/>
- ESP-IDF Programming Guide: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/index.html>

# 1. ESP32 Specification

**This section shows ESP32 specifications.**

**1.1 ESP32 by Thai Easy Electronics**

**1.2 ESP-WROOM-32 Chip**

**1.3 Pin Layout**

**1.4 ESP32 Resources**

# 1.1 ESP32 by Thai Easy Electronics

A feature-rich MCU with integrated Wi-Fi and Bluetooth connectivity for a wide-range of applications.

**ESP-WROOM-32 Chip: (2.4 GHz Wi-Fi and Bluetooth Combo SoC)**

**The RESET button**

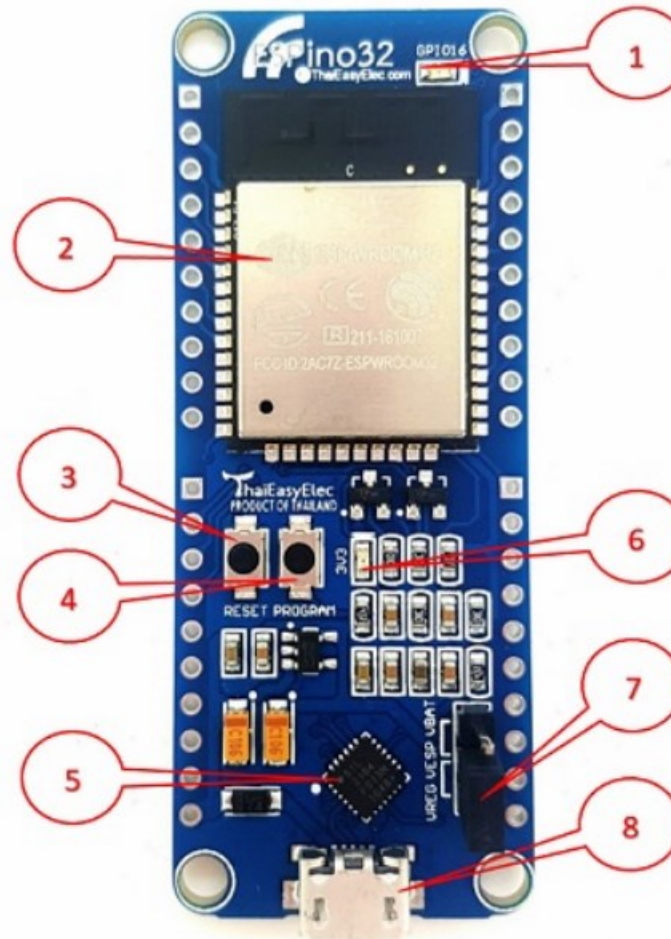
**The PROGRAM button**

**Chip CP2104: USB to serial**

**An LED connected to Pin 16**

**LED showing the power supply 3.3 V**

**The port Micro USB connected to a computer to upload an Arduino program**



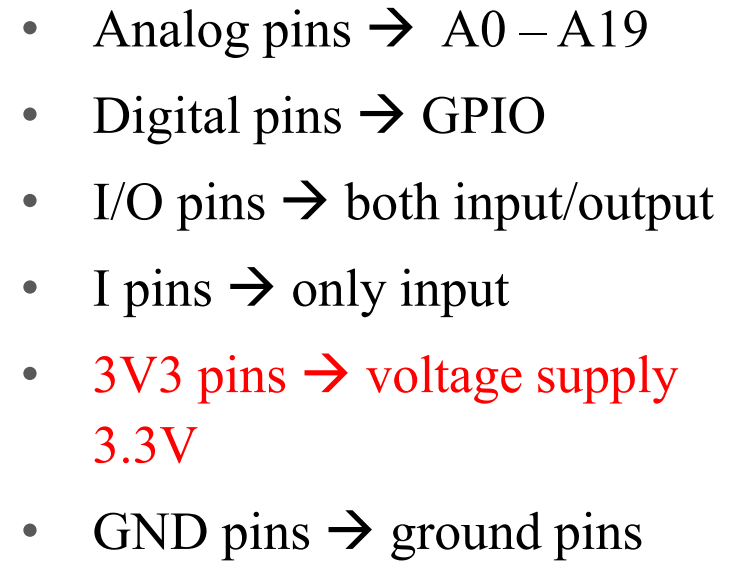
## 1.2 ESP-WROOM-32 Chip

ESP32-WROOM-32 is a powerful, generic Wi-Fi+BT+BLE MCU module that targets a wide variety of applications.

- ESP32 Dual-Core Tensilica LX6 Microcontroller 32-bit with the speed 240 MHz
- SRAM 520 KB
- Flash Memory 2 MB (16 Mbit)
- IEEE 802.11 b/g/n (frequency band: 2.4GHz)
- Bluetooth Dual Mode (Classic and BLE)
- 2.2V to 3.6V Operating Voltage
- On-Board PCB Antenna
- 32 GPIOs (3 x UARTs, 3 x SPI, 2 x I2S, 12 x ADC, 2 x DAC, 2 x I2C, )
- On-Board Sensors: temperature sensor, hall sensor, capacitive touch interface
- 



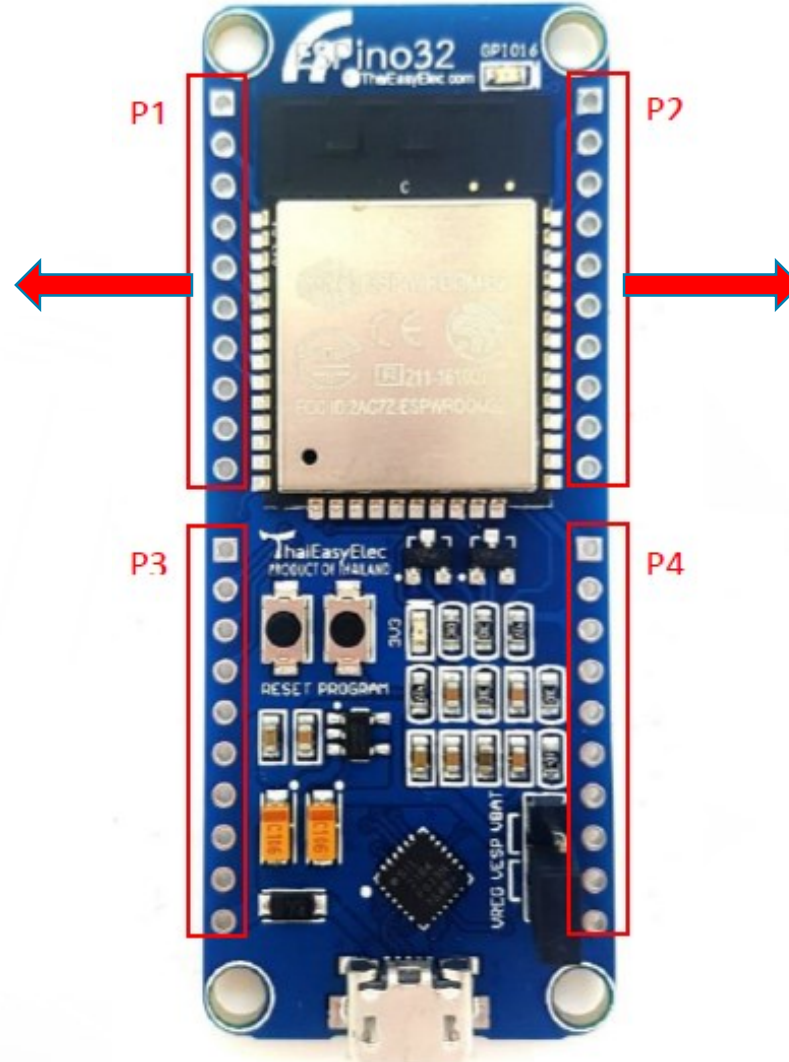




Not recommended for other uses

# 1.3 Pin Functions

| P1: Pin Label | Function                      |
|---------------|-------------------------------|
| GND           | Ground                        |
| 3V3           | 3.3 Volts                     |
| EN            | Enable                        |
| A0 / 36       | ADC1_CH0 / GPIO36             |
| A3 / 39       | ADC1_CH3 / GPIO39             |
| A4 / T9 / 32  | ADC1_CH4 / TOUCH9<br>/ GPIO32 |
| A5 / T8 / 33  | ADC1_CH5 / TOUCH8<br>/ GPIO33 |
| A6 / 34       | ADC1_CH6 / GPIO34             |
| A7 / 35       | ADC1_CH7 / GPIO35             |
| A10 / T0 / 4  | ADC2_CH0 / TOUCH0<br>/ GPIO4  |

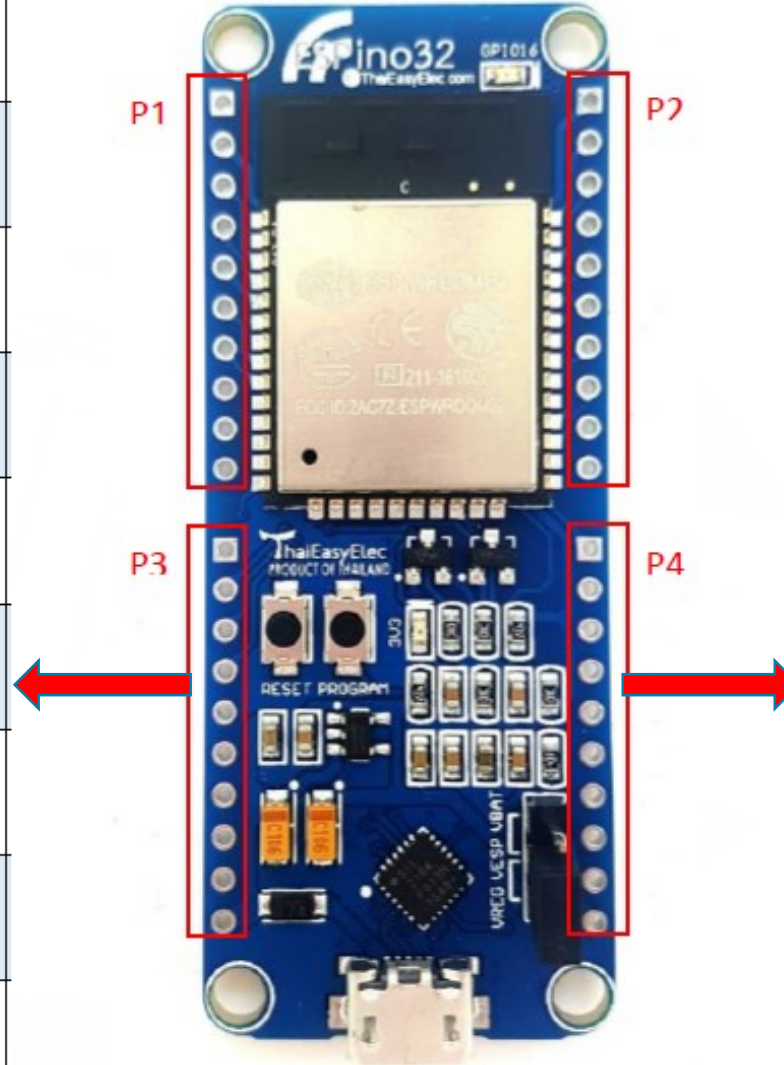


| P2: Pin Label | Function          |
|---------------|-------------------|
| 3V3           | 3.3 volts         |
| GND           | Ground            |
| SCL/22        | I2C SCL / GPIO22  |
| SDA/21        | I2C SDA / GPIO21  |
| Tx            | U0TXD / GPIO3     |
| Rx            | U0RXD / GPIO1     |
| 17            | GPIO17            |
| 16            | GPIO16            |
| SCK / 18      | SPI SCK / GPIO18  |
| MISO / 19     | SPI MISO / GPIO19 |



# 1.3 Pin Functions

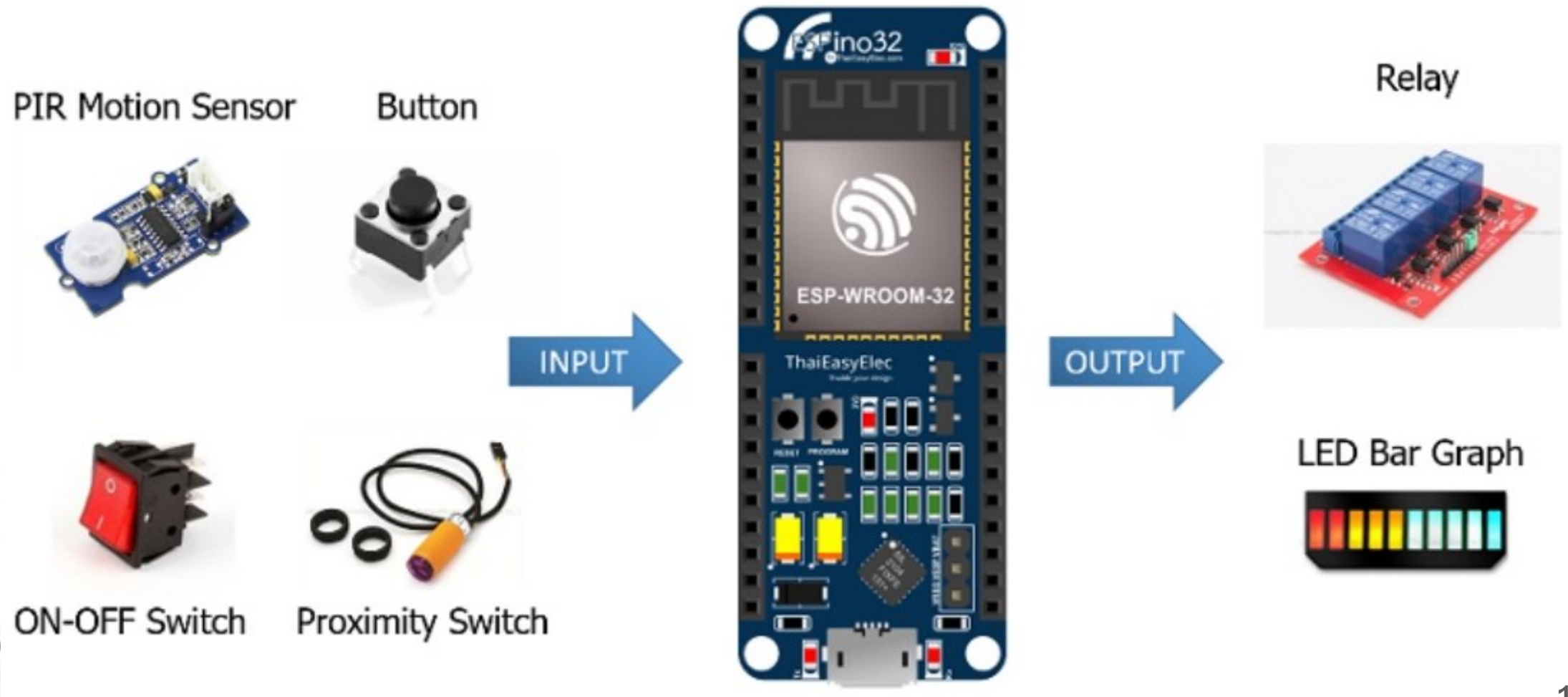
| P3: Pin Label   | Function                      |
|-----------------|-------------------------------|
| A11 / T1 / 0    | ADC2_CH1 / TOUCH1<br>/ GPIO0  |
| A12 / T2 / 2    | ADC2_CH2 / TOUCH2<br>/ GPIO2  |
| A13 / T3 / 15   | ADC2_CH3 / TOUCH3<br>/ GPIO15 |
| A14 / T4 / 13   | ADC2_CH4 / TOUCH4<br>/ GPIO13 |
| A15 / T5 / 12   | ADC2_CH5 / TOUCH5<br>/ GPIO12 |
| A16 / T6 / 14   | ADC2_CH6 / TOUCH6<br>/ GPIO14 |
| A17 / T7 / 27   | ADC2_CH7 / TOUCH7<br>/ GPIO27 |
| A18 / DAC1 / 25 | ADC2_CH8 / DAC_1 /<br>GPIO25  |
| A19 / DAC2 / 26 | ADC2_CH9 / DAC_2 /<br>GPIO26  |
| Vin             | Vin                           |



| P4: Pin Label | Function          |
|---------------|-------------------|
| MOSI / 23     | SPI MOSI / GPIO23 |
| SS / 5        | SPI SS / GPIO5    |
| SD2 / 9       | SD_DATA2 / GPIO9  |
| SD3 / 10      | SD_DATA3 / GPIO10 |
| CMD / 11      | SD_CMD / GPIO10   |
| CLK / 6       | SD_CLK / GPIO6    |
| SD0 / 7       | SD_DATA0 / GPIO7  |
| SD1 / 8       | SD_DATA1 / GPIO8  |
| GND           | Ground            |
| VBAT          | VBAT              |

# 1.3 Pin Layout

The microcontroller needs these pins to read data from input devices and to control output devices.



# 1.4 ESP32 Resources

- Thai Easy Elect: <https://blog.thaieasyelec.com/espino32-article-table-of-contents/>
- Last Minute Engineers: <https://lastminuteengineers.com/esp32-arduino-ide-tutorial/>
- Random Nerd Tutorials: <https://randomnerdtutorials.com/getting-started-with-esp32/>

## **2. Arduino IDE Setup and Uploading**

**This section consists of**

**2.1 Arduino IDE Setup**

**2.2 Uploading your Arduino program**

## 2.1 Arduino IDE Setup

### 1. Required Software: Arduino IDE

1. To download the Arduino IDE, go to website:

<https://www.arduino.cc/en/Main/Software>

2. Choose the one that is suitable to your computer:

#### DOWNLOAD OPTIONS

**Windows** Win 10 and newer, 64 bits

**Windows** MSI installer

**Windows** ZIP file

**Linux** AppImage 64 bits (X86-64)

**Linux** ZIP file 64 bits (X86-64)

**macOS** Intel, 10.15: "Catalina" or newer, 64 bits

**macOS** Apple Silicon, 11: "Big Sur" or newer, 64 bits

3. For Windows OS, there are 2 choices for Arduino installation:
  - If you choose Windows Installer, Arduino will automatically install all components (need the Internet).
  - If you choose Windows ZIP file, all necessary components are already in this ZIP file.



## 2.1 Arduino IDE Setup

### 2. Arduino IDE Installation by Windows ZIP File

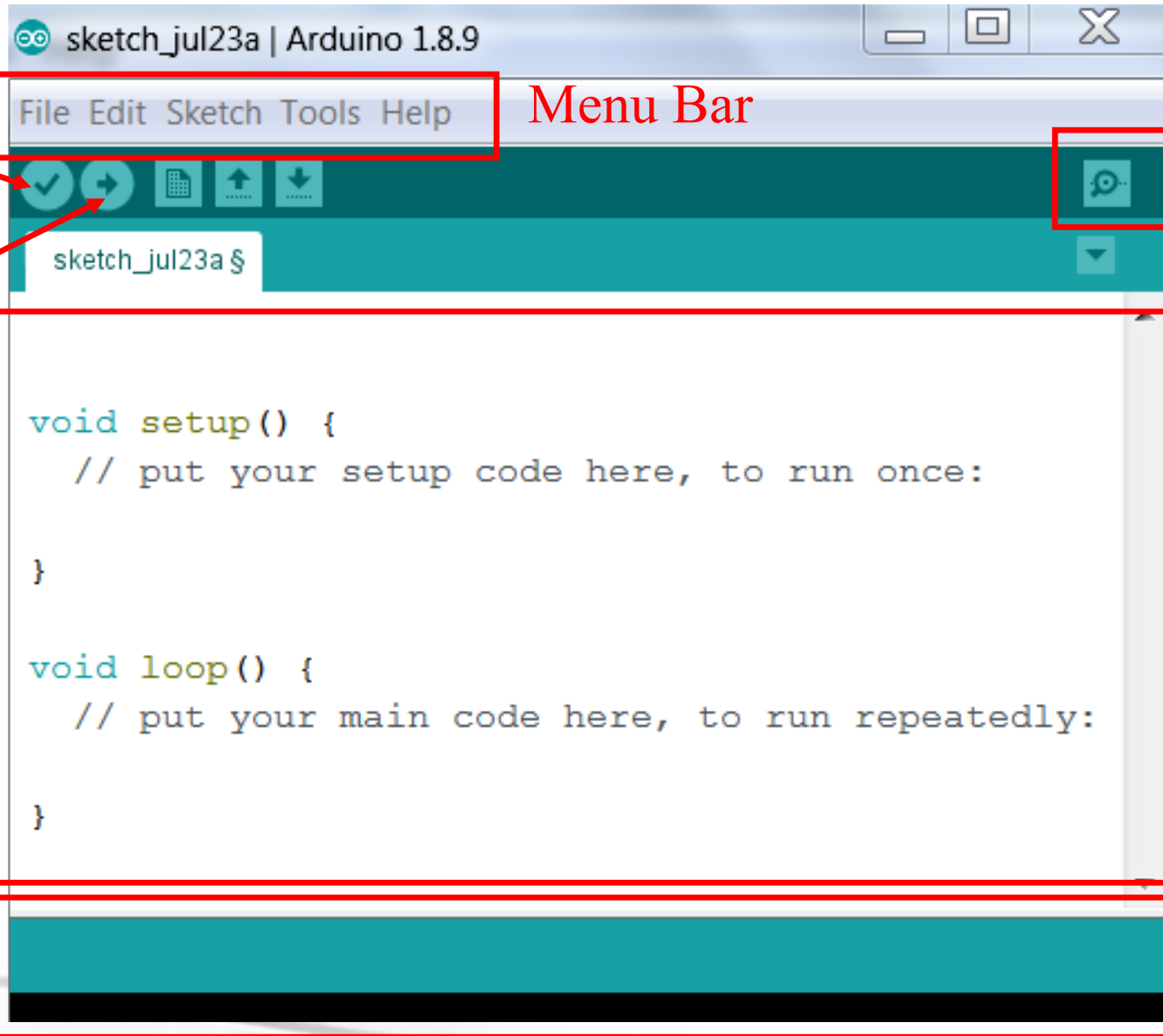
1. Unzip the file: Windows ZIP File to your computer (assume Drive C:).
2. You will get a folder arduino.
3. To run the Arduino IDE, **double click on “*arduino.exe*”** in the folder arduino.
4. If the program asks to install some components else, answer "Yes".

## 2.1 Arduino IDE Setup

### 3. Arduino IDE Interface

Verify  
Button

Upload  
Button



Serial Monitor button

Your Arduino Code

Information Window

## 2.1 Arduino IDE Setup

### 4. USB Driver

1. The USB driver that we need is called “*CP210x USB to UART Bridge VCP driver*”.
2. You can download it from this link:

<https://www.silabs.com/developers/usb-to-uart-bridge-vcp-drivers?tab=downloads>

|                                 |                       |
|---------------------------------|-----------------------|
| CP210x Universal Windows Driver | v11.5.0<br>12/31/2025 |
| CP210x VCP Mac OSX Driver       | v6.0.3<br>5/30/2025   |

3. Choose a suitable version for your OS:
  - 3.1. CP210x Universal Windows Driver: For Windows, download and unzip this file.
    - In the unzipped folder, to install, run either the file CP210xVCPInstaller\_x64.exe or the file CP210xVCPInstaller\_x86 (depending on your computer's platform).
  - 3.2. CP210x VCP Mac OSX Driver: For Mac OS, download and install this file.

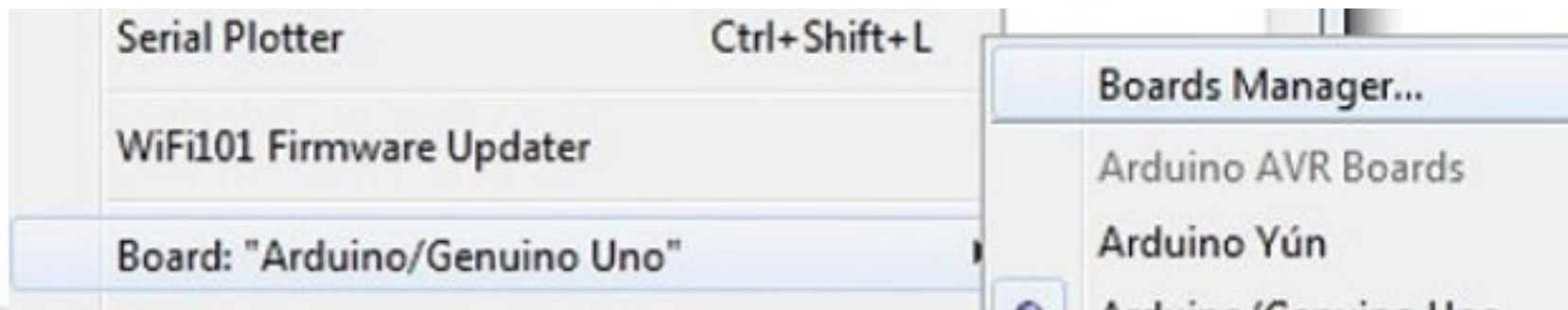
## 2.1 Arduino IDE Setup

### 5. Add ESP32 Board to Arduino IDE

1. Start the Arduino IDE by running “*Arduino.exe*”.
2. On the menu bar, select: *File > Preferences*.
3. In the “*Additional Boards Manager URLs*”, fill in  
[https://dl.espressif.com/dl/package\\_esp32\\_index.json](https://dl.espressif.com/dl/package_esp32_index.json)



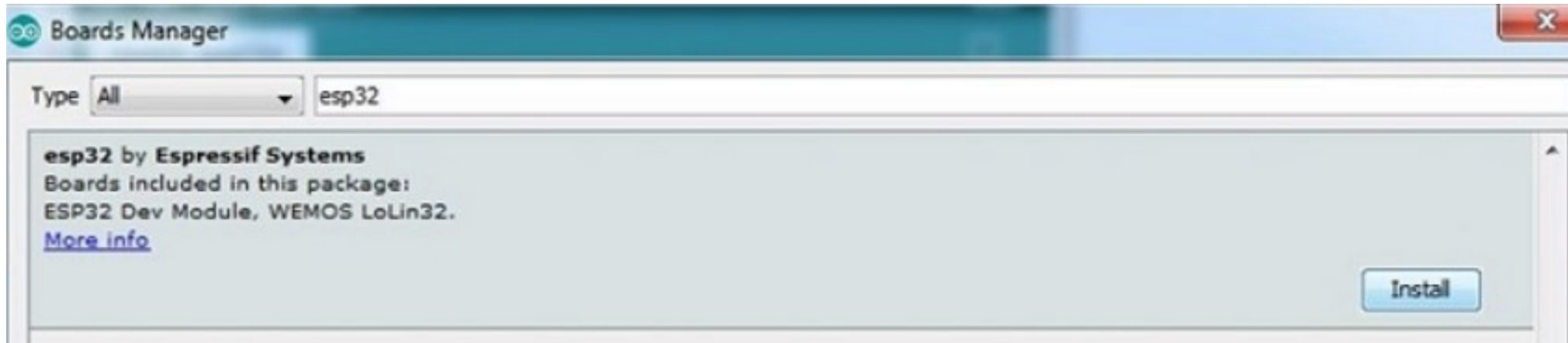
4. Then, click “*OK*”.
5. On the menu bar, select: *Tools > Board: > Boards Manager...*



## 2.1 Arduino IDE Setup

### 5. Add ESP32 Board to Arduino IDE

6. On the board manager, fill in: *esp32*
7. You will see “*esp32 by Espressif Systems*”, select it to install.



8. Finally, on the menu bar, select *Tools > Board > ThaiEasyElec's ESPino32*.
9. Now, you are ready to program the ThaiEasyElec's ESPino32.



## 2.2 Uploading Your Arduino Program

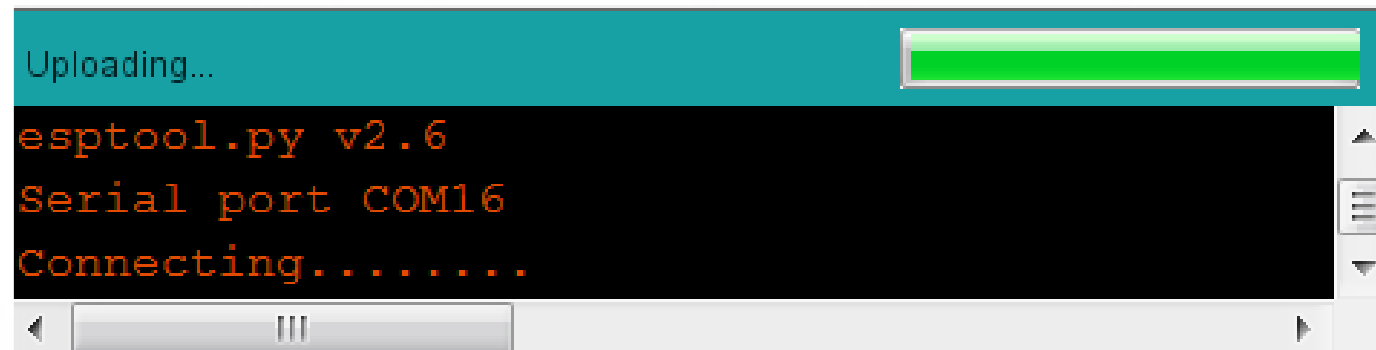
1. Write the following code on your Arduino IDE (this code will print Hello World on the serial monitor):

```
int a=0;
void setup() {
    Serial.begin(115200);
}
void loop() {
    a=a+1;
    Serial.println(a);
    delay(3000);
}
```

The baud rate of ESP32 must be 115200  
(not 9600)

## 2.2 Uploading Your Arduino Program

2. Connect ESP32 to your computer via a USB port.
3. On the menu bar of the Arduino IDE:
  - Select *Tools > Board: > ThaiEasyElect's ESPino32*
  - Select *Tools > Port > Your\_Arduino\_Com\_Port*
4. To upload your program to ESP32, click the *upload* icon.
5. On the information window (at the bottom of the Arduino IDE), whenever you see “*Connecting...*” as shown below, you have to press the *program* button on the ESP32.

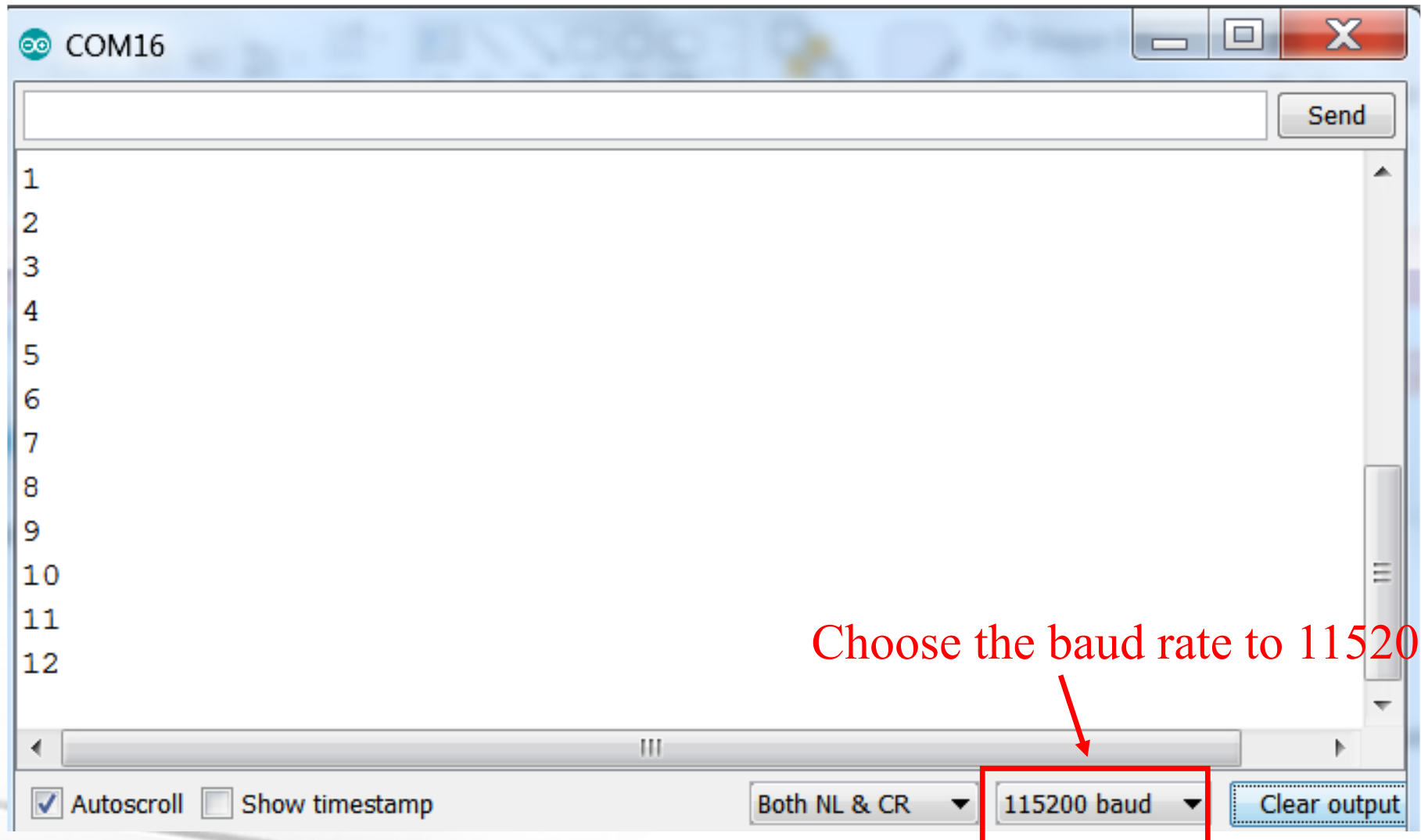


```
Uploading...
esptool.py v2.6
Serial port COM16
Connecting.....
```

6. If uploading the program is successful, you will see “*Done uploading*”.

## 2.2 Uploading Your Arduino Program

7. To open the serial monitor, click on the serial-monitor button.



# **3. Basic Features of ESP32**

**This section consists of**

**3.1 Digital Inputs and Outputs**

**3.2 Analog Inputs**

**3.3 Analog Outputs using PWM**

**3.4 Built-in Touch Sensors**

**3.5 Built-in Hall Effect Sensor**

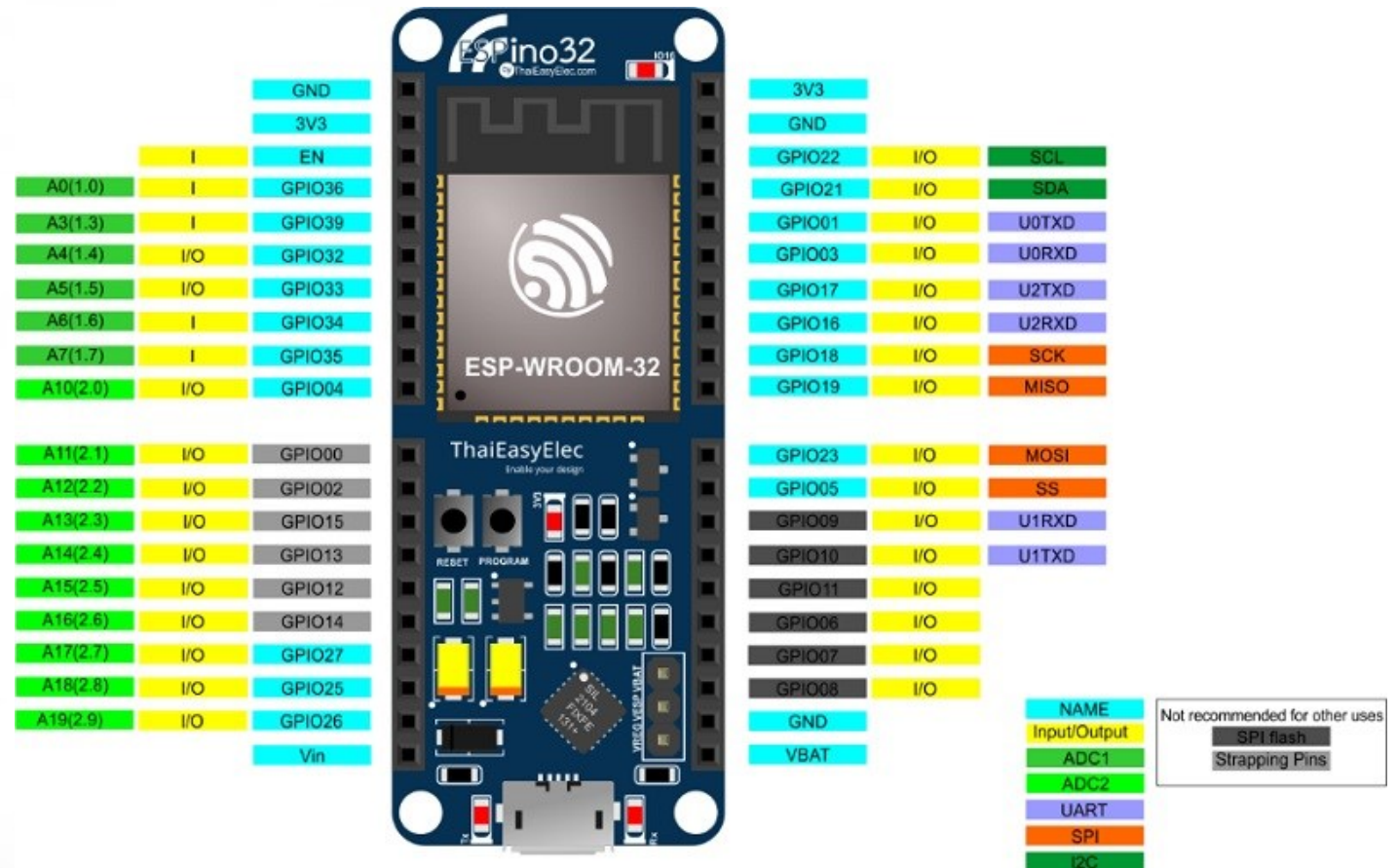
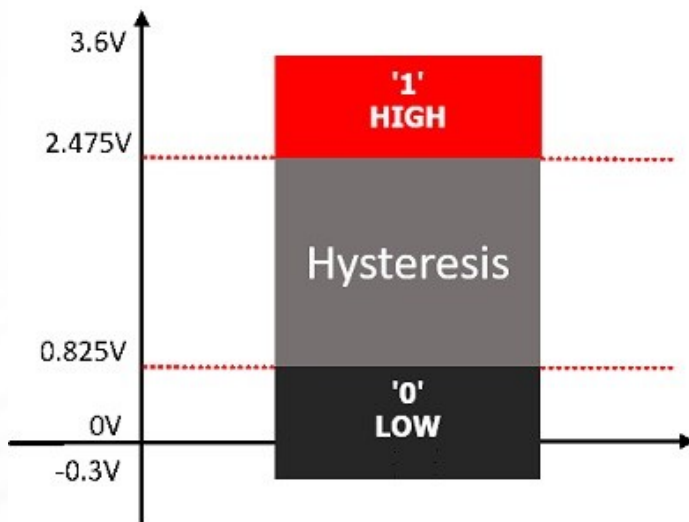
**3.6 Software Reset**

# 3.1 Digital Inputs and Outputs

## 1. Pins

We can read a digital value and control a digital device via the GPIO pins. ESP32 can read a digital input (0 or 1) and send a digital output (0 or 1) via the GPIO pins.

- Each pin can be referred in an Arduino program by its **GPIO number**. For example, the **GPIO22** pin is referred as **22** in the Arduino program.



# 3.1 Digital Inputs and Outputs

## 2. Send a Digital Value to a Digital Pin

There are 2 steps for sending.

- 1) **Set the pin mode.** Before we can use any digital pin, we have to declare the pin mode (INPUT or OUTPUT) of that pin. In general, we declare it in the *setup()* function. To use a digital pin as an output pin, we use this command:

*pinMode(pin, OUTPUT);*

where *pin* is the GPIO number.

- 2) **Send a digital value.** To send a digital value either 0 (LOW) or 1 (HIGH) to the pin declared above, we use this command:

*digitalWrite(pin, value);*

where the *value* is 0 or 1 (we can also use LOW or HIGH).

Note that:

- The value “1” means that this pin will supply the HIGH voltage (around 3.3 V).
- The value “0” means that this pin will supply the LOW voltage (around 0 V).



## 3.1 Digital Inputs and Outputs

### 3. Read a Digital Value from a Digital Pin

There are 2 steps for reading.

1) **Set the pin mode.** To use a digital pin as an input pin, we use this command:

*pinMode(pin, INPUT);*

where pin is the GPIO number. In general, we declare it in the *setup()* function

2) **Read a digital value.** To read a digital value either 0 (LOW) or 1 (HIGH) to the pin declared above, we use this command:

*X = digitalRead(pin);*

where a digital value (0 or 1) read from this pin will be stored in a variable X.

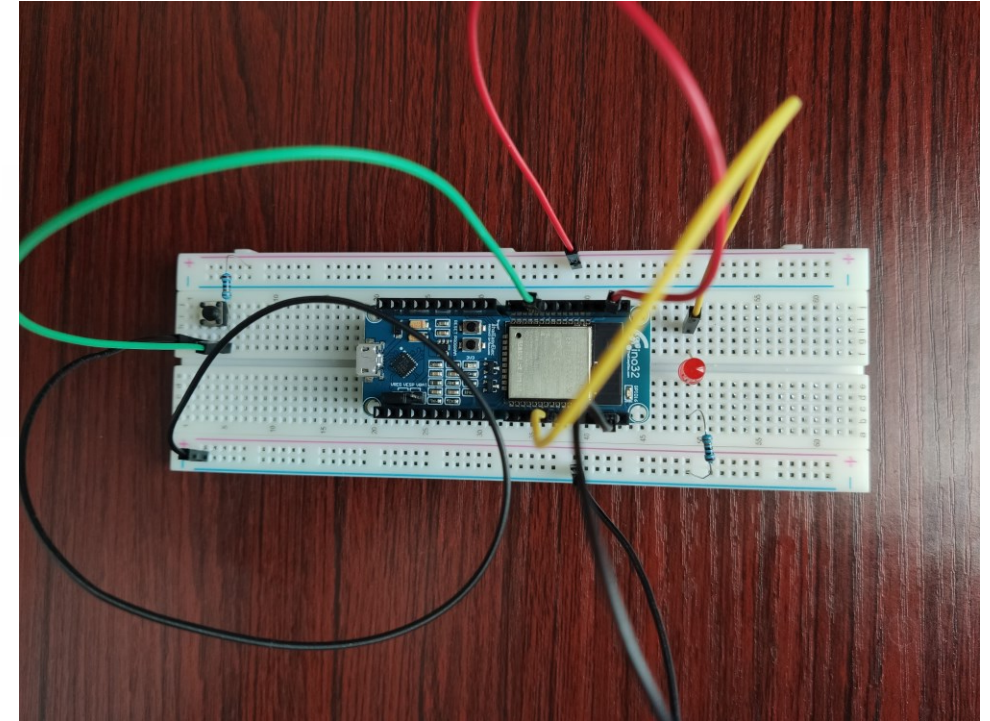
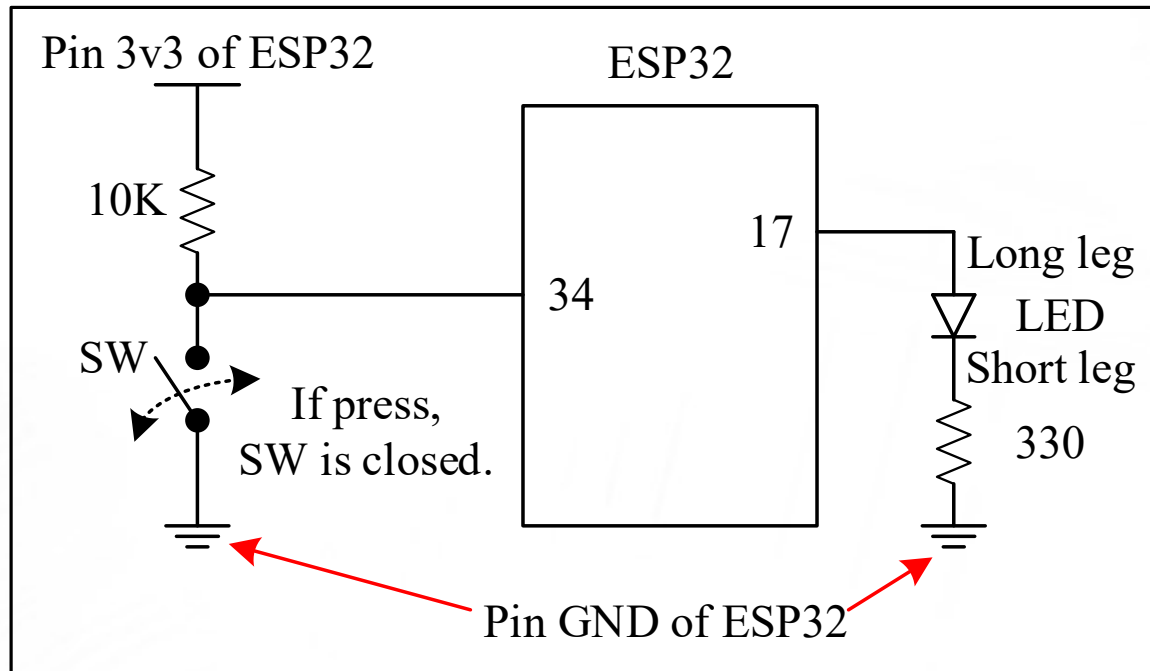
Note that:

- $X=1$  means that the input voltage to this pin is the HIGH voltage (around 3.3 V).
- $X=0$  means that the input voltage to this pin is the LOW voltage (around 0 V).

# 3.1 Digital Inputs and Outputs

## Example 1

We connect a button and an LED to ESP32 as shown below.



Then, **we write an Arduino code to turn on/off the LED by pressing the switch:**

- Press the switch → LED on;
- Not press the switch → LED off.

# 3.1 Digital Inputs and Outputs

## Example 1 (Digital\_InOut.ino)

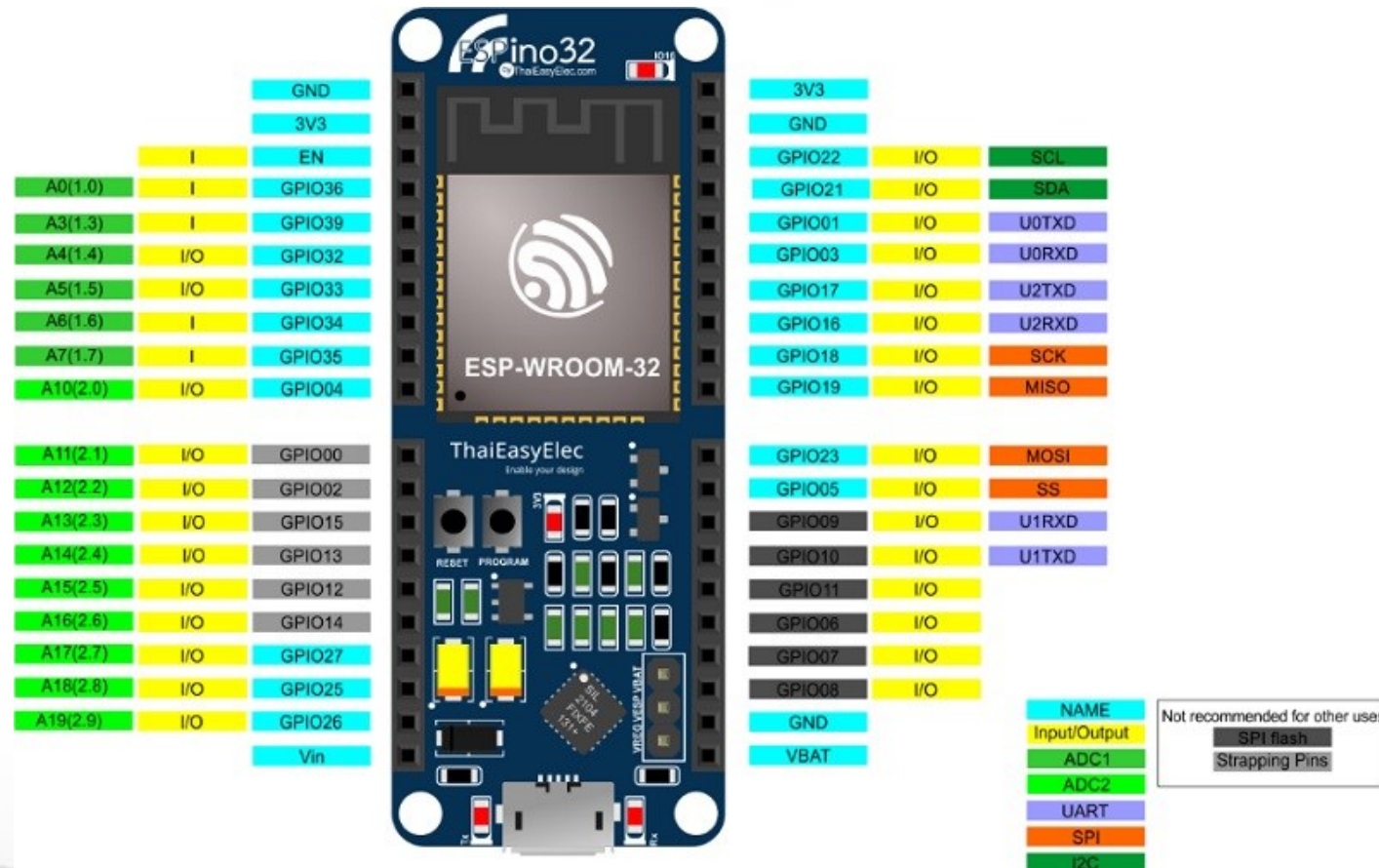
```
1 // set pin numbers
2 const int buttonPin = 34; // the push button pin
3 const int ledPin = 17;    // the LED pin
4
5 // variable for storing the pushbutton status
6 int buttonState = 0;
7
8 void setup() {
9     Serial.begin(115200);
10    // initialize the pushbutton pin as an input
11    pinMode(buttonPin, INPUT);
12    // initialize the LED pin as an output
13    pinMode(ledPin, OUTPUT);
14 }
15
```

```
16 void loop() {
17     // read the state of the pushbutton value
18     buttonState = digitalRead(buttonPin);
19     Serial.println(buttonState);
20     // check if the pushbutton is pressed.
21     // if it is, the buttonState is LOW
22     if (buttonState == LOW) {
23         // turn LED on
24         digitalWrite(ledPin, HIGH);
25     } else {
26         // turn LED off
27         digitalWrite(ledPin, LOW);
28     }
29 }
```

## 3.2 Analog Inputs

### 1. Pins

We can read an equivalent analog value (voltage) from an input device connected to an analog pin (labeled as A0 – A19). The input voltage 0 – 3.3 volt will be mapped to the integers 0 – 4095 (the value that ESP32 will show you), where 0 is 0 volt and 4095 is 3.3 volt.



## 3.2 Analog Inputs

### 2. Read an Analog Value from an Analog Pin

To read an analog input (from an inputted device) in the ESP32 using the Arduino IDE, we use the *analogRead()* function, whose syntax is as follows:

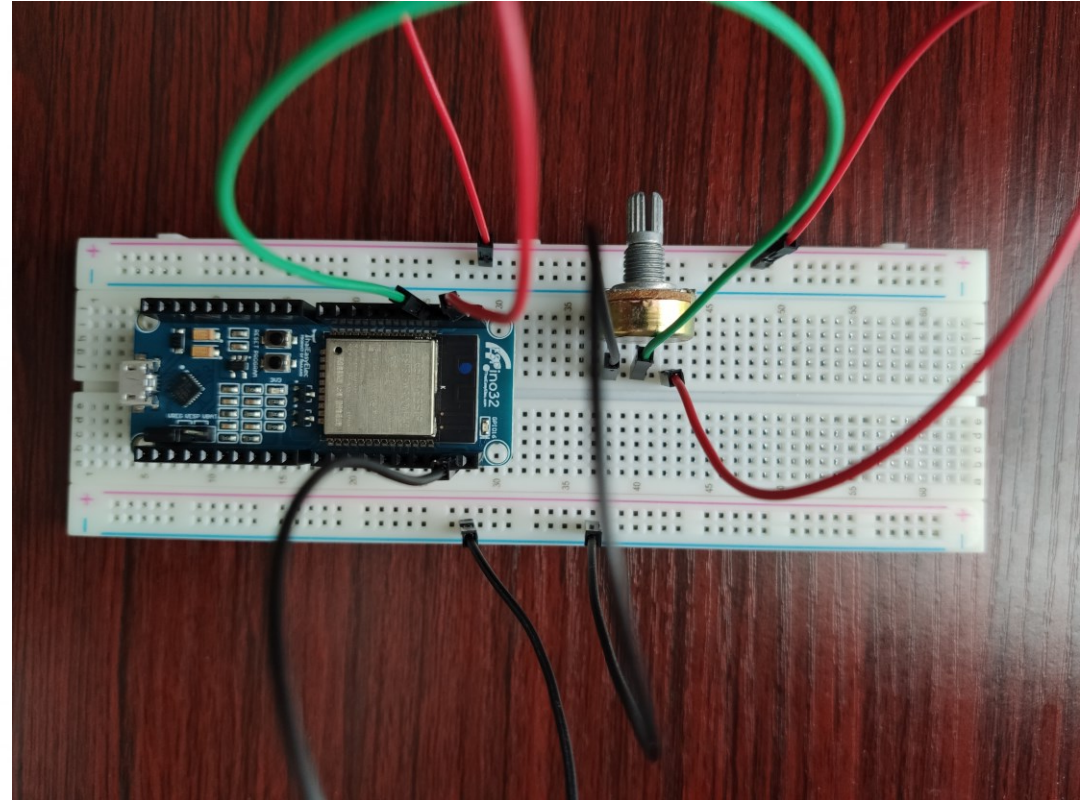
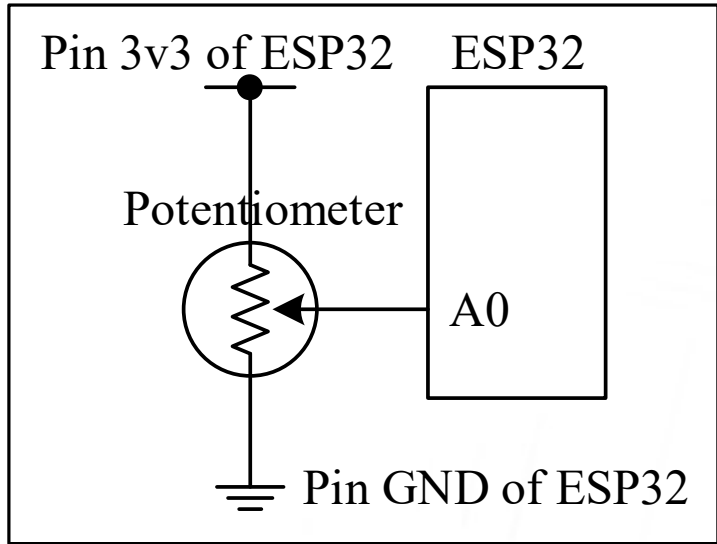
$$X = \text{analogRead}(\text{pin});$$

where pin is the **name of an ESP32 analog pin (A0 – A19)** and X is a variable storing a returned value between 0 – 4095.



## 3.2 Analog Inputs Example 2

We connect a potentiometer to ESP32 as shown below.



Then, we write an Arduino code to read the value from this pin and show it on the serial monitor while we rotate the potentiometer.



## 3.2 Analog Inputs

### Example 2 (Analog\_In.ino)

```
1 const int potPin = A0; //Potentiometer pin
2
3 // Variable for storing the potentiometer value
4 int Value = 0;
5
6 void setup() {
7     Serial.begin(115200);
8     delay(1000);
9 }
10
11 void loop() {
12     // Reading potentiometer value
13     Value = analogRead(potPin);
14     Serial.println(Value);
15     delay(1000);
16 }
```

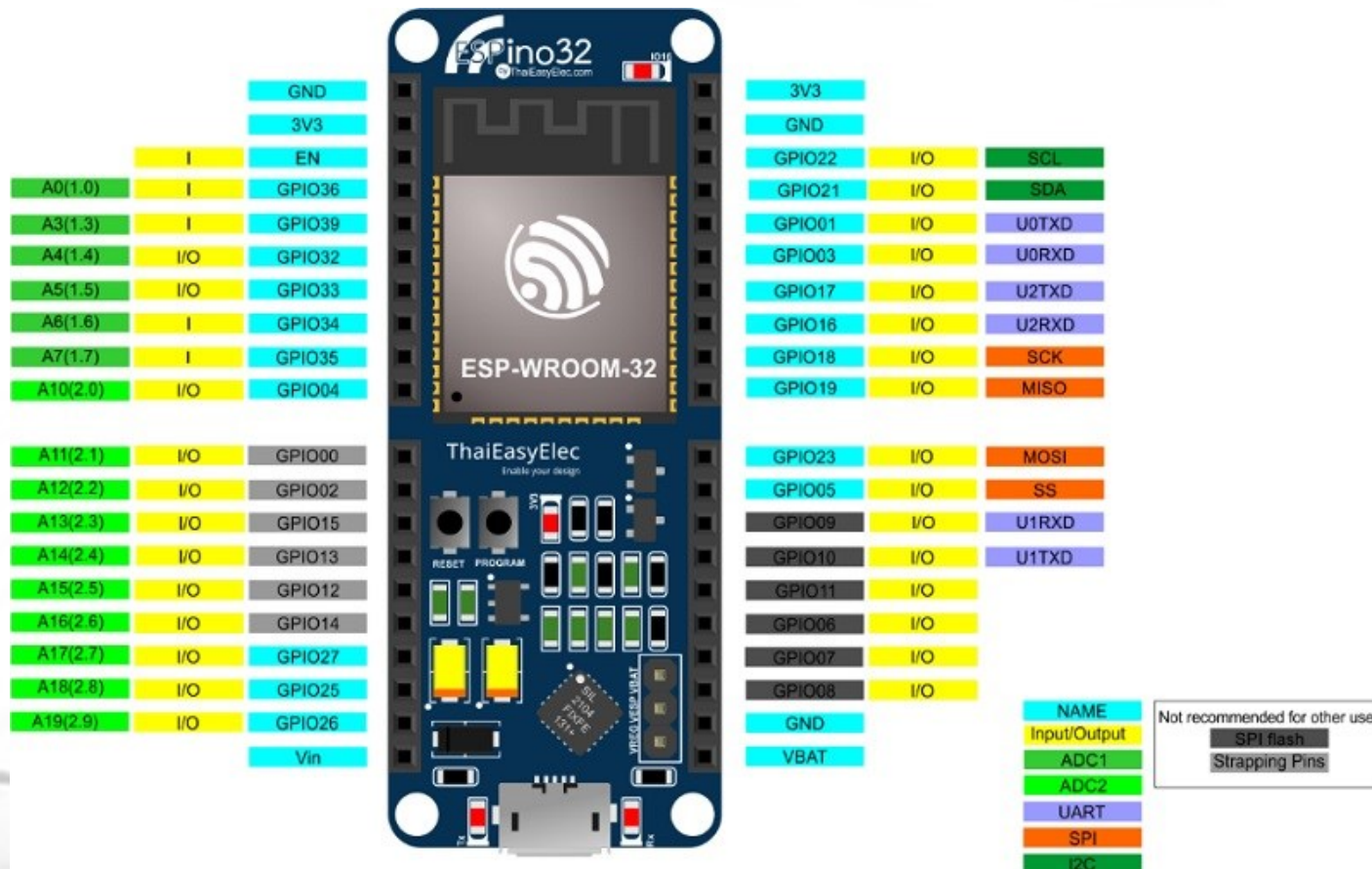


# 3.3 Analog Outputs using PWM

## 1. Pins

There are several methods to control an output analog device. Here, we will PWM method to send out (from the ESP32) an equivalent analog voltage via the PWM pins.

- PWM pins: GPIO 0-19, 21-23, 25-27, 32-33.
- The values sent out will be 0 – 255 which are equivalent to 0 – 3.3 volt.



# 3.3 Analog Outputs using PWM

## 2. Send an Analog Value to a PWM Pin

We have the following 3 steps.

- 1) We have to declare these **three variables at the beginning of the Arduino code**:
  - a) We need to choose a PWM channel. There are 16 channels from 0 to 15.
  - b) Then, we need to set the PWM signal frequency. For an LED, a frequency of 5000 Hz is fine to use.
  - c) We need to set the signal's duty cycle resolution. For ESP32, it is 8 since we use 8-bit resolution.

For example,

```
int ledChannel = 0;  
int freq = 5000;  
int resolution = 8;
```

## 3.3 Analog Outputs using PWM

### 2. Send an Analog Value to a PWM Pin

2) In the **setup()** function,

a) we declare this setup:

*ledcSetup(ledChannel, freq, resolution);*

b) we will map the PWM pin and the PWM channel. As a result, the PWM signal will be sent to this PWM pin. We will use this function:

*ledcAttachPin(PWMpin, ledChannel);*

where PWMpin is the GPIO number of a PWM pin and channel is the variable defined in Step 1.

3) To send a PWM signal to the PWM pin, we control the channel by using the following function:

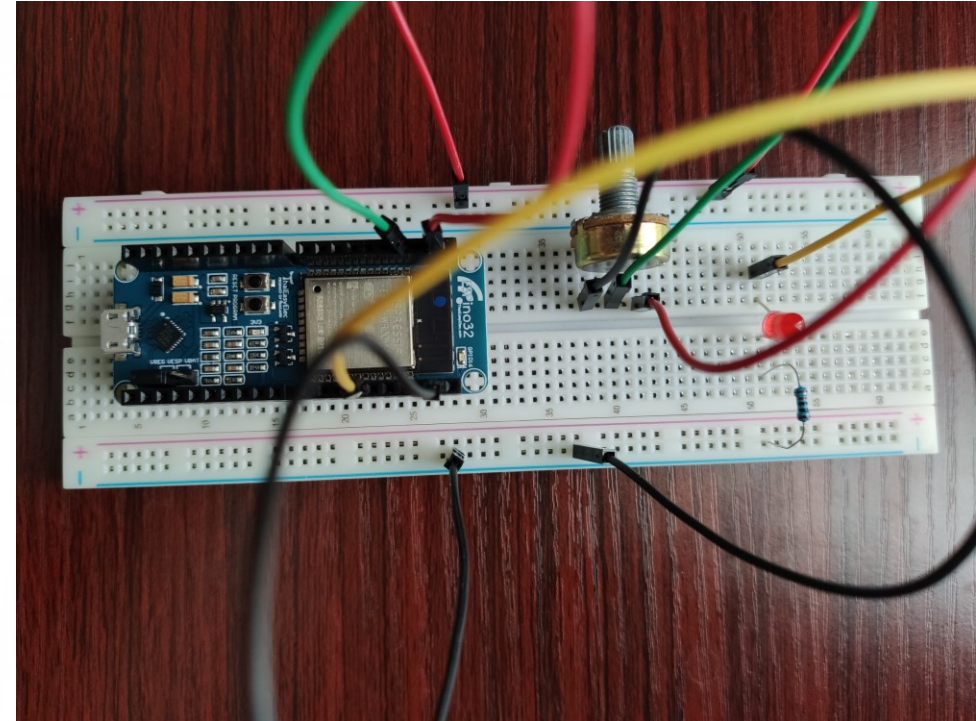
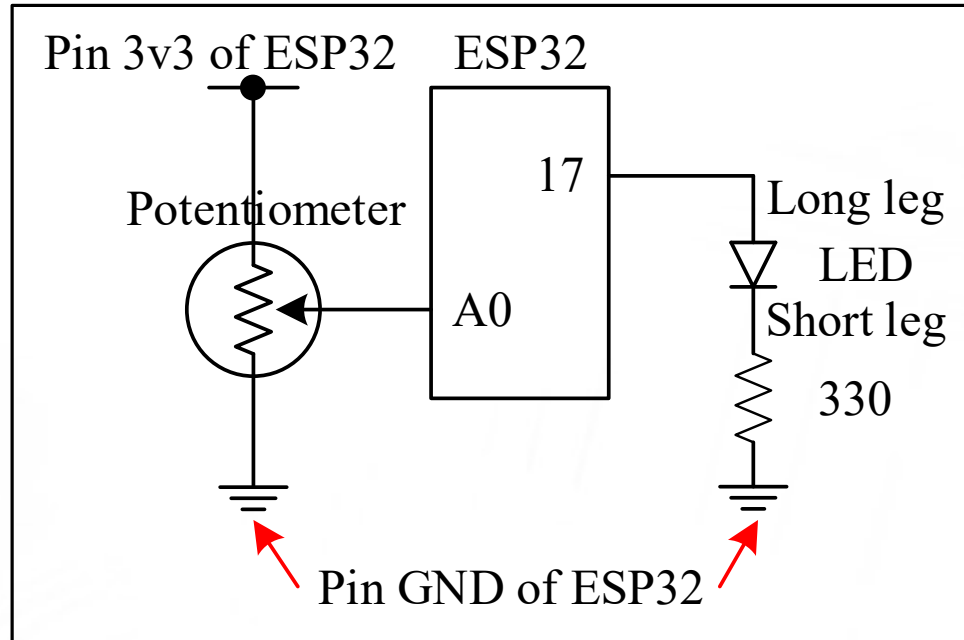
*ledcWrite(ledChannel, dutycycle)*

where channel is the variable defined in Step 1 and the duty cycle is a value between 0 – 255 (0 is 0 volt and 255 is 3.3 volt).

## 3.3 Analog Outputs using PWM

### Example 3

We connect a potentiometer and an LED to ESP32 as shown below.



Then, we write an Arduino code such that we can adjust the brightness of the LED by rotating the potentiometer.



# 3.3 Analog Outputs using PWM

## Example 3 (Analog\_PWM.ino)

```
1 const int potPin = A0; //Potentiometer pin
2 const int ledPin = 17; // LED pin
3
4 // setting PWM properties
5 const int freq = 5000;
6 const int ledChannel = 0;
7 const int resolution = 8;
8
9 // Variable for storing the potentiometer value
10 int potValue = 0;
11 int ledValue = 0;
12
13 void setup() {
14     // configure LED PWM functionalitites
15     ledcSetup(ledChannel, freq, resolution);
16
17     // attach the channel to the GPIO to be controlled
18     ledcAttachPin(ledPin, ledChannel);
19 }
20
```

Step 1: Declare variables

Step 2: Set up

```
21 void loop() {
22     // Reading potentiometer value
23     potValue = analogRead(potPin);
24     ledValue = map(potValue, 0, 4095, 0, 255);
25
26     // changing the LED brightness with PWM
27     ledcWrite(ledChannel, ledValue);
28 }
```

Step 3: Send the command

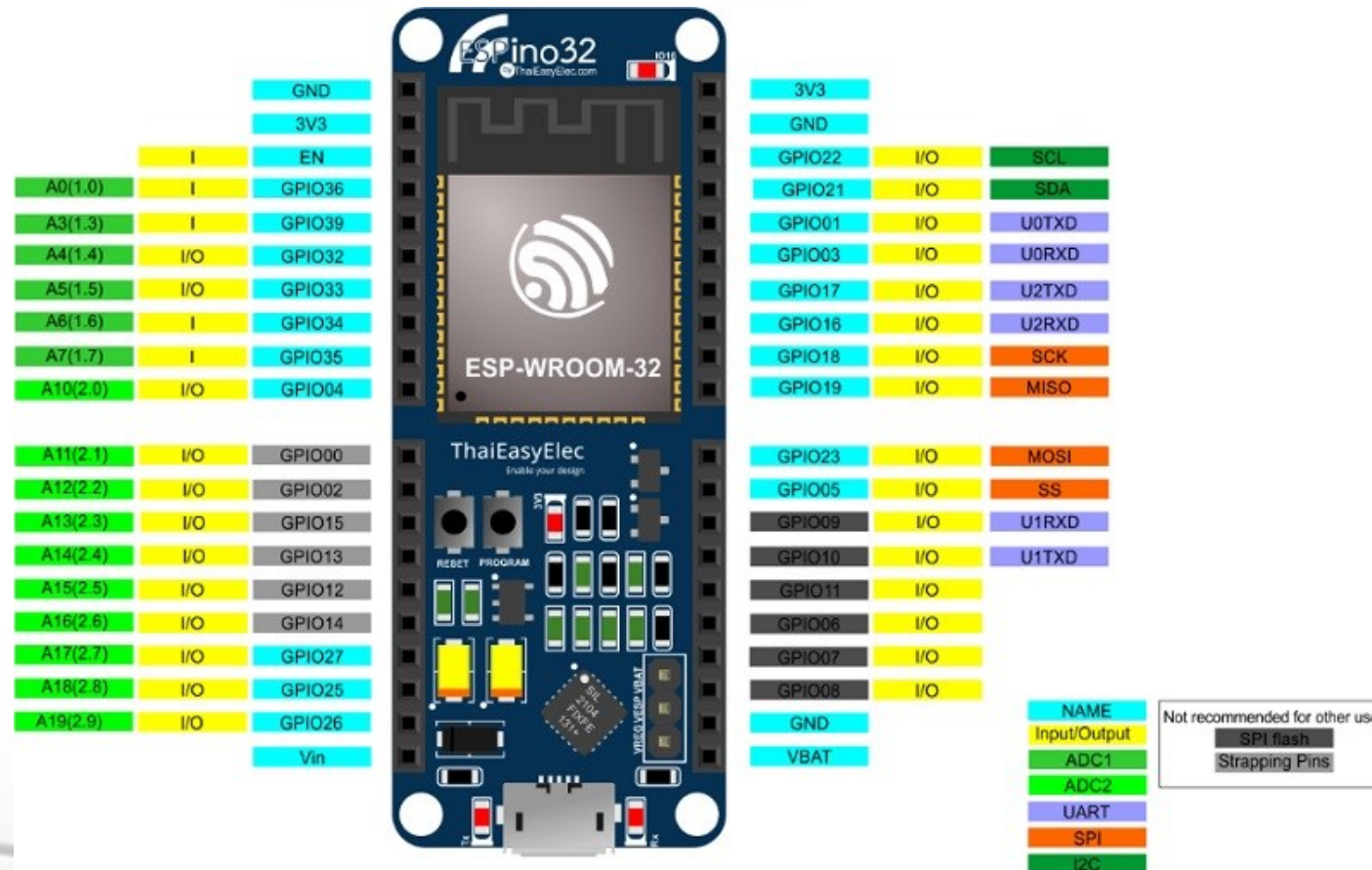


# 3.4 Built-in Touch Sensors

## 1. Pins

The ESP32 has 10 capacitive touch GPIOs. These GPIOs can sense variations in anything that holds an electrical charge, like the human skin. So they can detect variations induced when touching a GPIO pin with a finger.

- These pins are the following GPIO pins: 0, 2, 4, 12, 13, 14, 15, 27, 32, 33.



## 3.4 Built-in Touch Sensors

### 2. Command to Read from a Touch Sensor

To read a value from a touch-sensor pin, we use the following command:

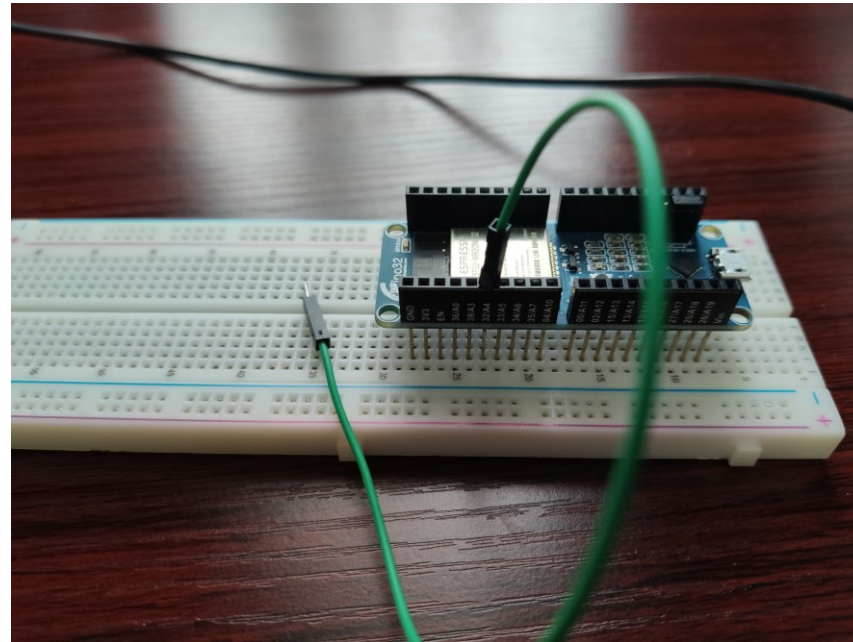
$$X = \text{touchRead}(\text{pin});$$

where pin is the GPIO number of a touch-sensor pin and **X stores an integer between 0 and 1023** returned from this command. When no one touch this pin, a high value will be returned. When we touch this pin, a low value will be returned.

## 3.4 Built-in Touch Sensors

### Example 4

We connect a wire to the pin GPIO32 (a touch-sensor pin) of ESP32 as shown below.



Then, we write an Arduino code to read the value from this pin and show on the serial monitor, every 1 second.

## 3.4 Built-in Touch Sensors

### Example 4 (Touch\_Pin.ino)

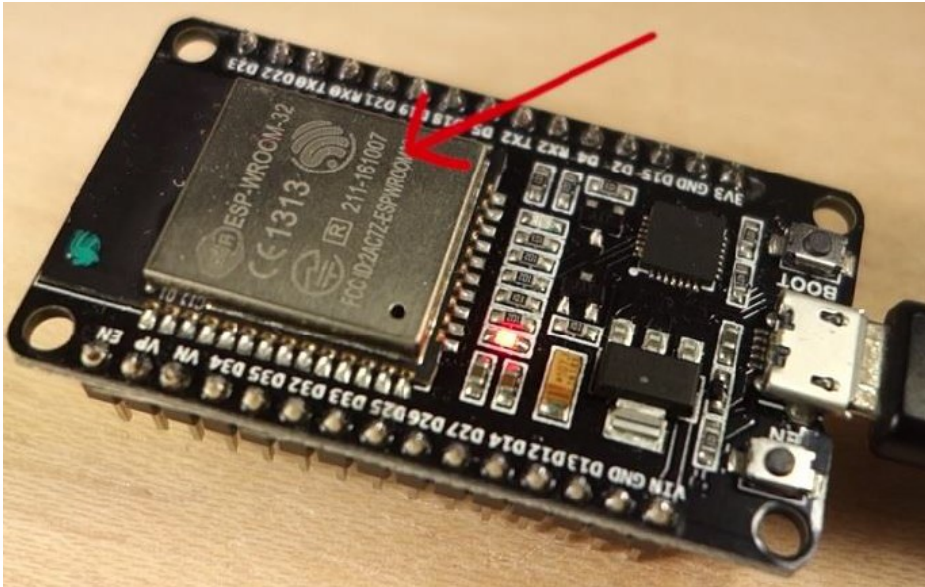
```
1 void setup() {  
2   Serial.begin(115200);  
3   delay(1000); // give me time to bring up serial monitor  
4   Serial.println("ESP32 Touch Test");  
5 }  
6  
7 void loop() {  
8   Serial.println(touchRead(32)); // Touch sensor from Pin 32  
9   delay(1000);  
10 }
```

The values given  
from the touch  
sensor when we  
touch the wire.



## 3.5 Built-in Hall Sensor Command

The ESP32 features a built-in hall effect sensor which is located behind the metal cap. A **hall effect sensor can detect variations in the magnetic field in its surroundings**. The greater the magnetic field, the greater the output voltage.



To read the value from the Hall sensor, we use the command:

$$X = \text{hallRead}();$$

where  **$X$  stores a returned integer between -500 and 500** from the built-in Hall sensor.

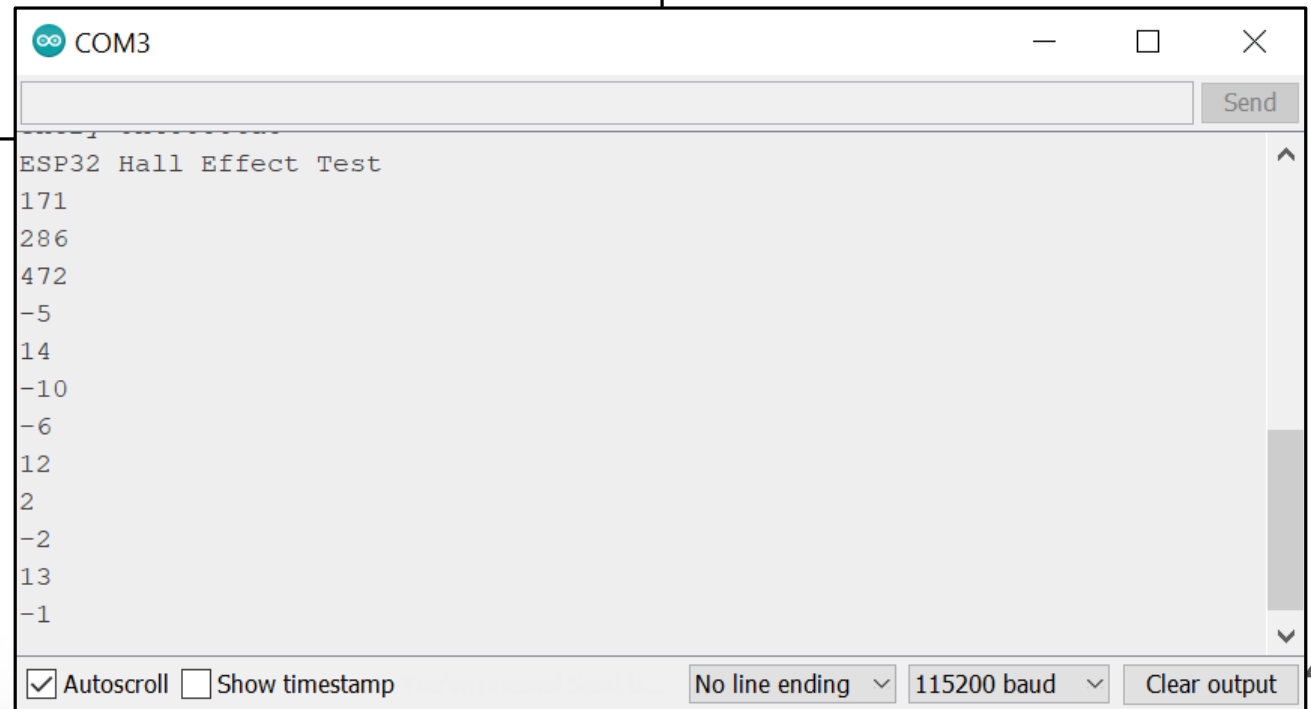


## 3.5 Built-in Hall Sensor

### Example 5 (Hall\_Effect.ino)

```
1 void setup() {  
2   Serial.begin(115200);  
3   delay(1000); // give me time to bring up serial monitor  
4   Serial.println("ESP32 Hall Effect Test");  
5 }  
6  
7 void loop() {  
8   Serial.println(hallRead()); // Read the hall effect  
9   delay(1000);  
10 }
```

Using the code here, ESP32 will read a value from the hall-effect sensor and shown on the serial monitor, every 1 second.



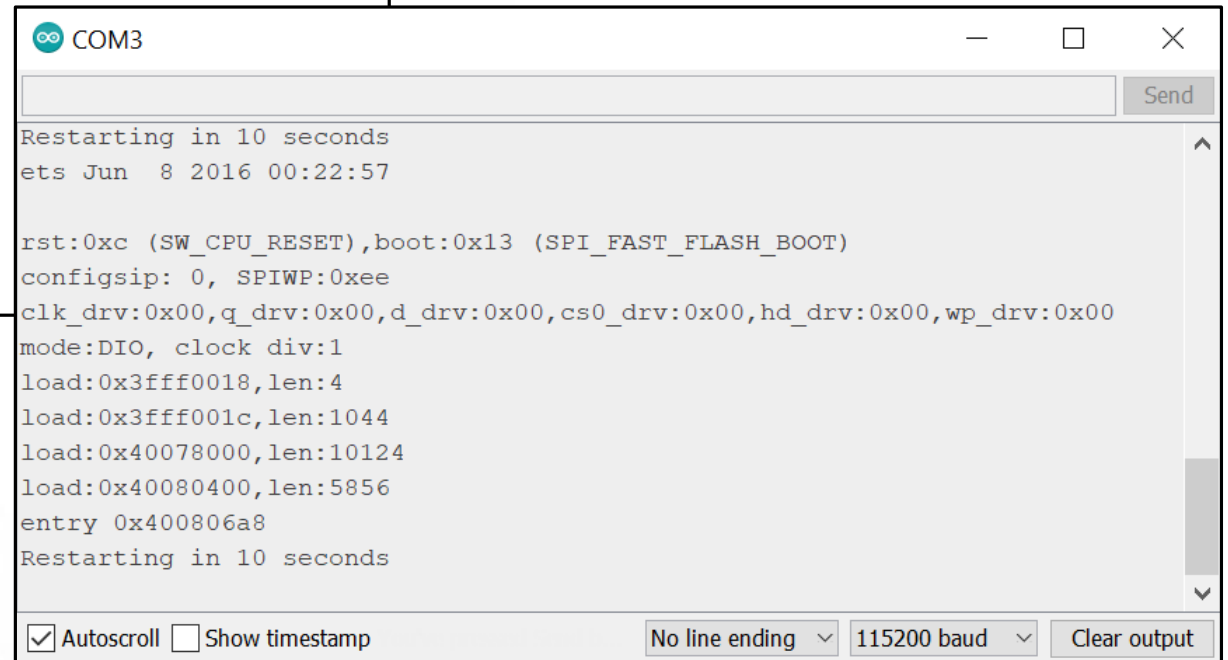
## 3.6 A Software Reset Command and Example

We can use the following command to **restart the Arduino program in ESP32**:

*ESP.restart();*

A code example (Soft\_Reset.ino) is shown below: **ESP32 will restart every 10 seconds.**

```
1 void setup() {  
2     Serial.begin(115200);  
3     Serial.println("Restarting in 10 seconds");  
4     delay(10000);  
5     ESP.restart();  
6 }  
7  
8 void loop() {  
9 }
```



COM3

Restarting in 10 seconds  
ets Jun 8 2016 00:22:57  
rst:0xc (SW\_CPU\_RESET),boot:0x13 (SPI\_FAST\_FLASH\_BOOT)  
config: 0, SPIWP:0xee  
clk\_drv:0x00,q\_drv:0x00,d\_drv:0x00,cs0\_drv:0x00,hd\_drv:0x00,wp\_drv:0x00  
mode:DIO, clock div:1  
load:0x3fff0018,len:4  
load:0x3fff001c,len:1044  
load:0x40078000,len:10124  
load:0x40080400,len:5856  
entry 0x400806a8  
Restarting in 10 seconds

☒ Autoscroll ☐ Show timestamp No line ending 115200 baud Clear output

# 4. WiFi Connection

**This section consists of**

**4.1 Roles of ESP32 in a WiFi Network**

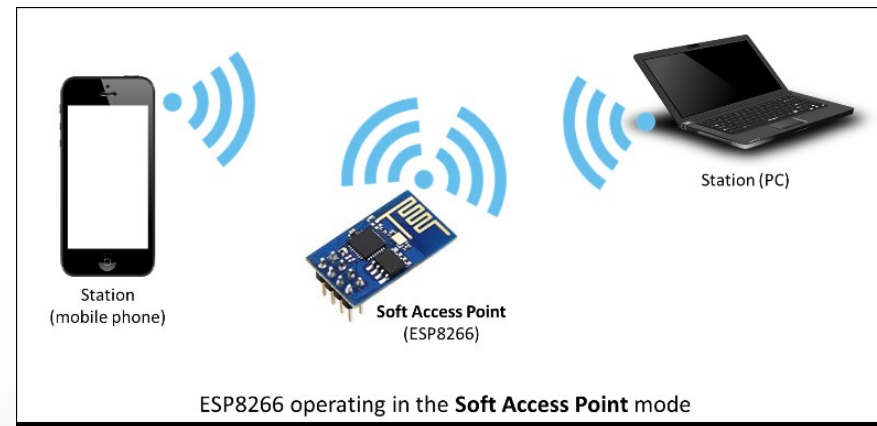
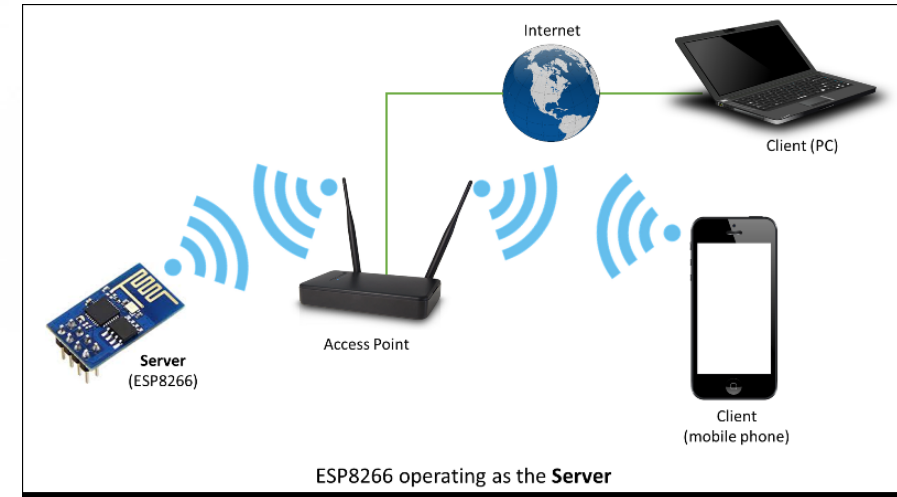
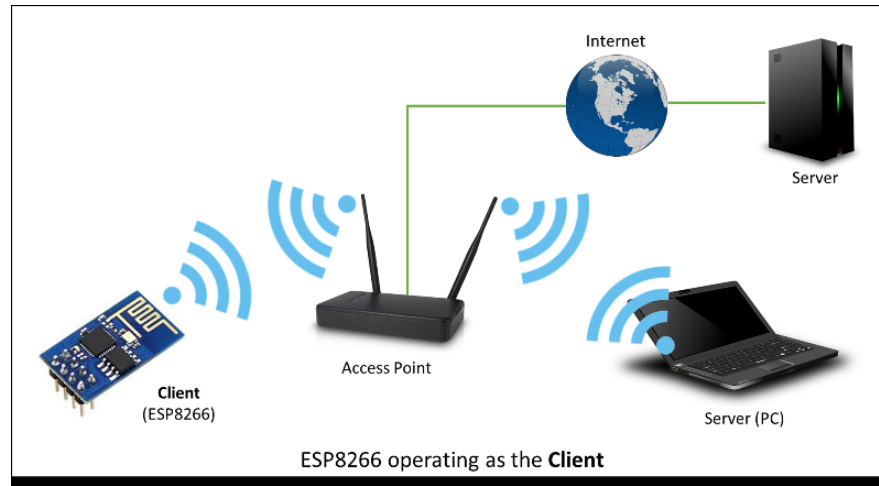
**4.2 Commands to Connect to a WiFi Network**

**4.3 Commands to Set up as a Local Soft Access Point**

**4.4 Getting Date and Time from an NTP Server**

# 4.1 Roles of ESP32 in a WiFi Network

ESP32 has a WiFi module which can communicate via a WiFi network. It can be set up as a **web client**, a **web server**, or a **local access point**.



## 4.2 Commands to Connect to a WiFi Network

- 1) To connect WiFi by using ESP32, we need to include the header file “*WiFi.h*” in our Arduino code.

*#include <WiFi.h>*

- 2) We have the following commands to set up and connect to a WiFi network.

| Function                                            | Explanation                                                                                                                  |
|-----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <i>WiFi.begin(“<b>ssid</b>”, “<b>password</b>”)</i> | ESP32 will try to connect to the access point <b>ssid</b> by using the password <b>password</b> (the password of this SSID). |
| <i>WiFi.localIP()</i>                               | This function returns ESP32’s IP address.                                                                                    |
| <i>WiFi.mode(WIFI_STA)</i>                          | Let this ESP32 work as a (WiFi) station.                                                                                     |



## 4.2 Commands to Connect to a WiFi Network

| Function             | Explanation                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>WiFi.status()</i> | <p>This function returns following codes to describe what is going on with Wi-Fi connection:</p> <ul style="list-style-type: none"><li>• 0 : WL_IDLE_STATUS when Wi-Fi is in process of changing between statuses,</li><li>• 1 : WL_NO_SSID_AVAIL in case configured SSID cannot be reached,</li><li>• 3 : WL_CONNECTED after successful connection is established,</li><li>• 4 : WL_CONNECT_FAILED if password is incorrect,</li><li>• 6 : WL_DISCONNECTED if module is not configured in station mode.</li></ul> |

## 4.2 Commands to Connect to a WiFi Network

### Example 6 (WiFi\_Connection.ino)

```
#include <WiFi.h>
```

→ The header file to connect to WiFi

```
void setup() {
```

```
  Serial.begin(115200);
```

→ Baud rate = 115200

```
  Serial.println();
```

```
  WiFi.mode(WIFI_STA);
```

```
  WiFi.begin("Your_SSID", "Your_Password");
```

```
  Serial.print("Connecting");
```

```
  while (WiFi.status() != WL_CONNECTED) {
```

```
    delay(500);
```

```
    Serial.print(".");
```

```
  }
```

```
  Serial.println();
```

```
  Serial.print("Connected, IP address: ");
```

```
  Serial.println(WiFi.localIP());
```

- Set the ESP32 in a station mode.
- Set ESP32 to connect to the SSID “Your\_SSID” by using the password “Your\_Password”.

→ ESP32 is trying to connect to the SSID.

→ Show the ESP32's IP address on the serial monitor.

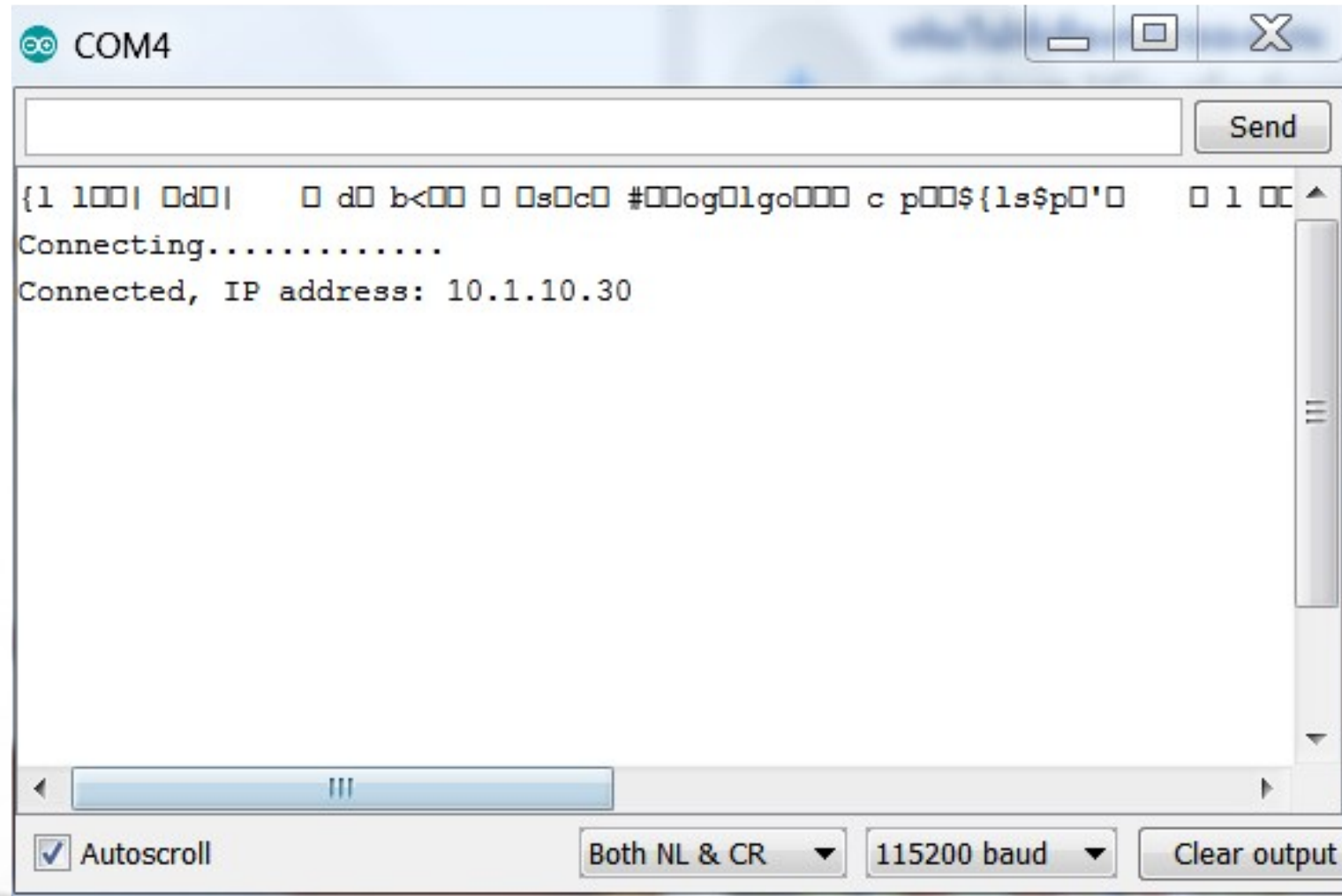
```
void loop() {
```

```
}
```

## 4.2 Commands to Connect to a WiFi Network

### Example 6

The results shown on the serial monitor:



```
{1 100| 0d| 0 d0 b<00 0 0s0c0 #00og0lgo000 c p00${ls$p0'0 0 1 00 ^  
Connecting.....  
Connected, IP address: 10.1.10.30
```

☒ Autoscroll    Both NL & CR ▼    115200 baud ▼    Clear output

## 4.3 Commands to Set up as a Local Access Point

- 1) To setup ESP32 as a local access point, we need to include the header file “*WiFi.h*” in our Arduino code.

*#include <WiFi.h>*

- 2) We have the following commands to set up ESP32 as a local access point.

| Function                                                | Explanation                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|---------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>WiFi.softAP(“ssid”, “password”, channel, hidden)</i> | <ul style="list-style-type: none"><li>• <i>ssid</i> - character string containing network SSID (max. 63 characters).</li><li>• <i>password</i> - optional character string with a password. For WPA2-PSK network it should be <u>at least 8 character long</u>. If not specified, the access point will be open for anybody to connect.</li><li>• channel - optional parameter to set Wi-Fi channel, from 1 to 13. Default channel = 1.</li><li>• hidden - optional parameter, if set to true will hide SSID.</li></ul> |
| <i>WiFi.softAPgetStationNum()</i>                       | This function returns the number of stations connecting to ESP32 (which is now an access point).                                                                                                                                                                                                                                                                                                                                                                                                                        |

## 4.3 Commands to Set up as a Local Access Point

### Example 7

The following code will set up ESP32 as a local access point and show the number of stations connecting to it. After uploading this code, ESP32 will work as a local access point and we can use our mobile phone to connect to it.

Key commands:

- *WiFi.softAP("Your\_SSID", "Your\_Password");* → To set ESP32 as an access point with SSID "Your\_SSID" and password "Your\_Password".

## 4.3 Commands to Set up as a Local Access Point

### Example 7 (WiFi\_SoftAccessPoint.ino)

```
#include <WiFi.h>
```

```
void setup() {  
  Serial.begin(115200);  
  Serial.println();  
  Serial.print("Setting soft-AP ... ");  
  boolean result = WiFi.softAP("Your_SSID", "Your_Password");  
  if(result == true) {  
    Serial.println("Ready");  
  } else {  
    Serial.println("Failed!");  
  }  
}
```

Set this ESP32 as an access point named  
*Your\_SSID* with the password *Your\_Password*



```
void loop() {  
  Serial.printf("Stations connected = %d\n", WiFi.softAPgetStationNum());  
  delay(3000);  
}
```

Return the number of stations  
connecting to this access point.

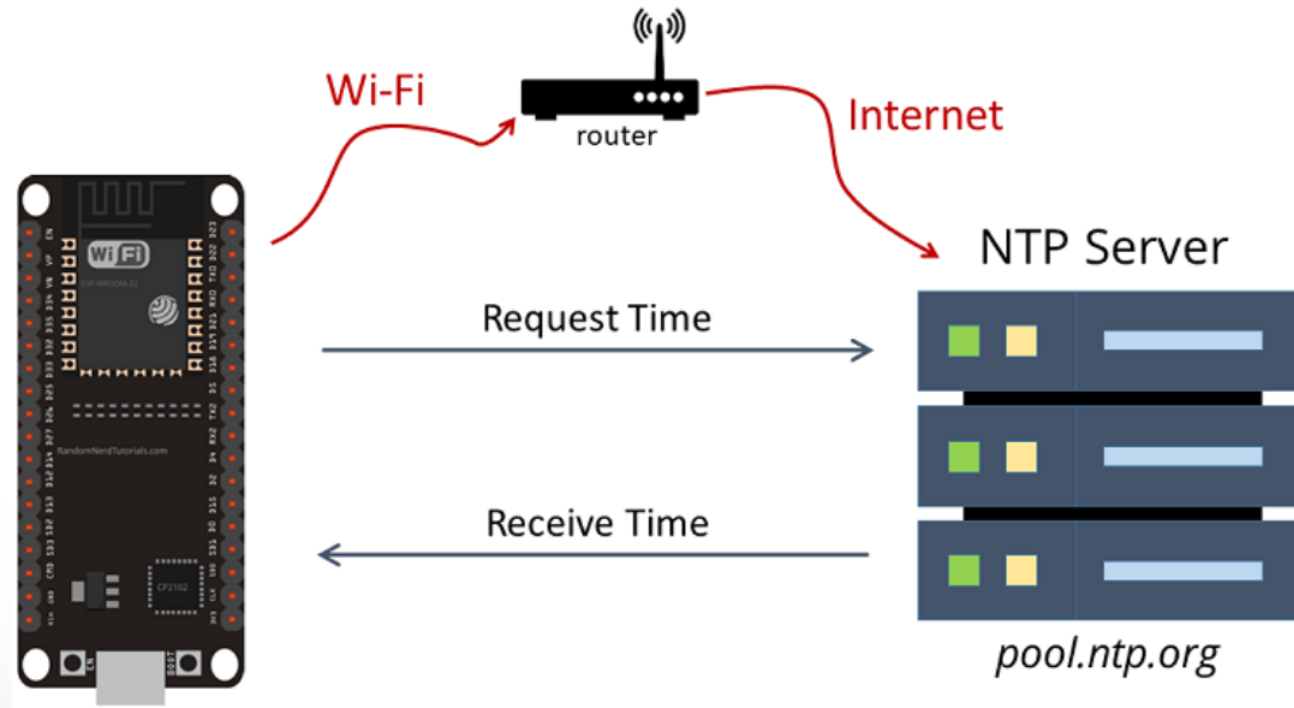




# 4.4 Getting Date and Time from an NTP Server

## 1. Introduction

- Getting date and time is especially useful in data logging to timestamp our readings.
- If ESP32 can access to the Internet, we can get date and time using Network Time Protocol (NTP) and **we do not need additional hardware (like an RTC clock)**.
- There are NTP servers like pool.ntp.org that anyone can use to request time as a client.
- In the following code, **the ESP32 is an NTP Client that requests time from an NTP Server (pool.ntp.org)**.



## 4.4 Getting Date and Time from an NTP Server

### Example 8 (DateTimerNTP.ino)

```
1 #include <WiFi.h>
2 #include "time.h"
3
4 const char* ssid      = "Your SSID";
5 const char* password  = "Your Password";
6
7 const char* ntpServer = "pool.ntp.org";
8 const long  gmtoffset_sec = 7*3600;
9 const int   daylightOffset_sec = 0;
10
11 void printLocalTime() {
12     struct tm timeinfo;
13     if(!getLocalTime(&timeinfo)){
14         Serial.println("Failed to obtain time");
15         return;
16     }
17     Serial.println(&timeinfo, "%A, %B %d %Y %H:%M:%S");
18 }
19
```

## 4.4 Getting Date and Time from an NTP Server

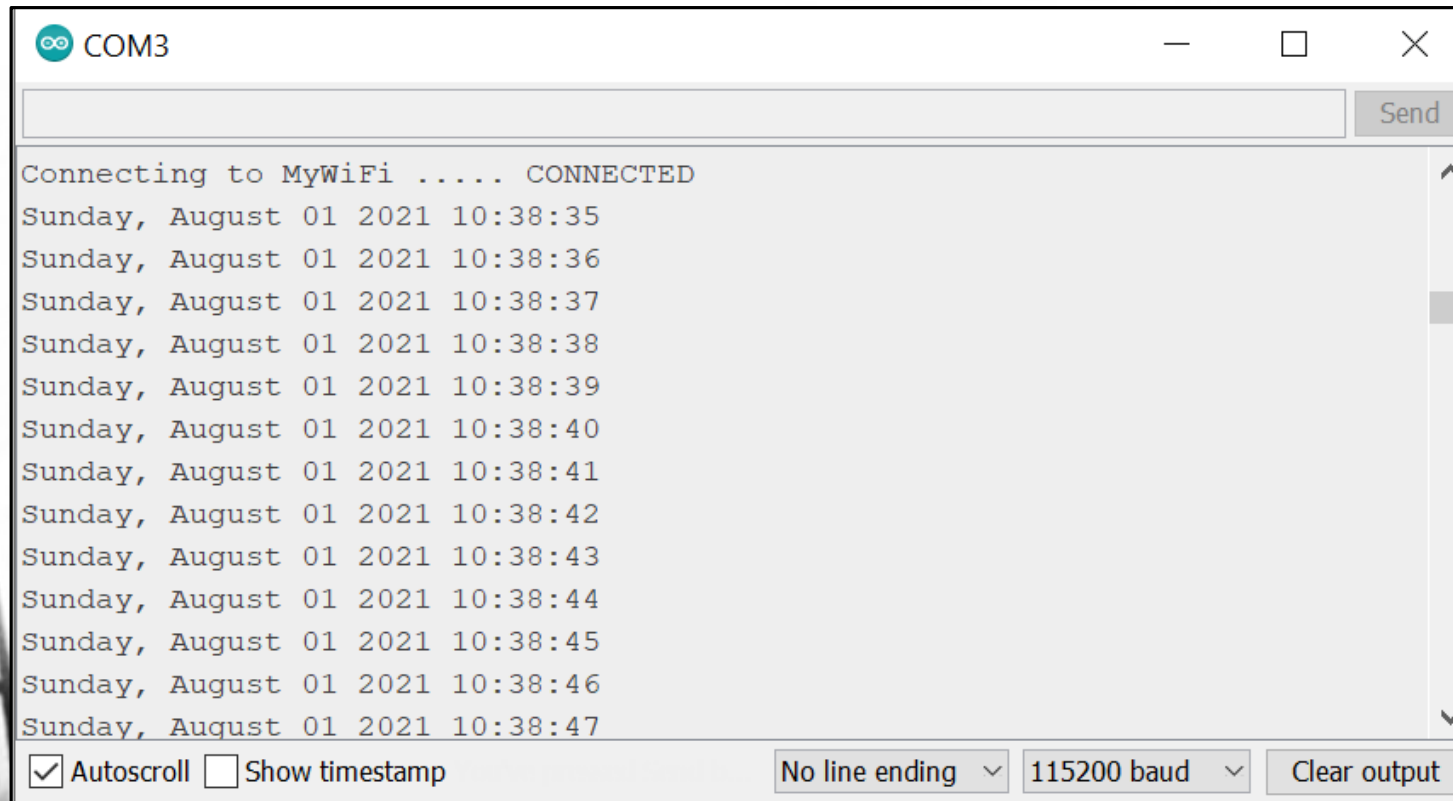
```
20 void setup() {  
21     Serial.begin(115200);  
22  
23     //connect to WiFi  
24     Serial.printf("Connecting to %s ", ssid);  
25     WiFi.begin(ssid, password);  
26     while (WiFi.status() != WL_CONNECTED) {  
27         delay(500);  
28         Serial.print(".");  
29     }  
30     Serial.println(" CONNECTED");  
31  
32     //init and get the time  
33     configTime(gmtOffset_sec, daylightOffset_sec, ntpServer);  
34     printLocalTime();  
35  
36     //disconnect WiFi as it's no longer needed  
37     WiFi.disconnect(true);  
38     WiFi.mode(WIFI_OFF);  
39 }  
40
```

## 4.4 Getting Date and Time from an NTP Server

### Example 8

```
41 void loop() {  
42     delay(1000);  
43     printLocalTime();  
44 }
```

The current date and time are shown on the serial monitor.



## 4.4 Getting Date and Time from an NTP Server

### Explanations

- 1) Once ESP32 is connected to the network, we initialize the NTP client using the *configTime()* function to get date and time from an NTP server.

```
32 //init and get the time
33 configTime(gmtOffset_sec, daylightOffset_sec, ntpServer);
```

This function needs three input parameters, which have been defined earlier:

```
7 const char* ntpServer = "pool.ntp.org";
8 const long  gmtOffset_sec = 7*3600;
9 const int    daylightOffset_sec = 0;
```

where *ntpServer* is the URL of an NTP server, *gmtOffset\_sec* is the UTC offset for our time zone in milliseconds (Thailand is +7 hours), and *daylightOffset* is the Daylight saving time (0 for Thailand).

## 4.4 Getting Date and Time from an NTP Server Explanations

2) The *printLocalTime()* custom function is to print current date & time.

```
11 void printLocalTime() {  
12     struct tm timeinfo;  
13     if(!getLocalTime(&timeinfo)) {  
14         Serial.println("Failed to obtain time");  
15         return;  
16     }  
17     Serial.println(&timeinfo, "%A, %B %d %Y %H:%M:%S");  
18 }
```

- In Line 13, the *getLocalTime()* function will get date and time from the NTP server and store them in the variable *timeinfo*.



## 4.4 Getting Date and Time from an NTP Server

### Explanations

- Line 17 prints the received date and time stored in *timeinfo* into to a readable format where

|    |                       |
|----|-----------------------|
| %A | returns day of week   |
| %B | returns month of year |
| %d | returns day of month  |
| %Y | returns year          |
| %H | returns hour          |
| %M | returns minutes       |
| %S | returns seconds       |

## 4.4 Getting Date and Time from an NTP Server

### Remark

- 1) We connect to the NTP server only one time at the beginning. **The time later is counted by the internal clock.**
- 2) If we want to store date or time in a variable, we use the *strftime()* function,
  - For example, we can copy the information about the hour that is on the *timeinfo* structure into the *timeHour* variable (declared to store up to 3 characters: 2 for two digit numbers and **1 for the terminating character**) as follows

```
char timeHour[3];  
strftime(timeHour,3, "%H", &timeinfo);  
Serial.println(timeHour);
```

- For example, for the week day, we need to create a char variable with a length of 10 characters because the longest day of the week contains 9 characters (Saturday).

```
char timeWeekDay[10];  
strftime(timeWeekDay,10, "%A", &timeinfo);  
Serial.println(timeWeekDay);  
Serial.println();
```

## 4.4 Getting Date and Time from an NTP Server

### Remark

3) More explanations can be found here:

<https://randomnerdtutorials.com/esp32-date-time-ntp-client-server-arduino/>

4) In addition, we can write an Arduino code to get Epoch/Unix Time. The Epoch Time (also known as Unix epoch, Unix time, POSIX time or Unix timestamp) is the number of seconds that have elapsed since January 1, 1970. The code example is shown here:

<https://randomnerdtutorials.com/epoch-unix-time-esp32-arduino/>

## **5. Advanced Topics to Explore More**

# 5. ESP32 Over-The-Air (OTA) Updates

## 1. What is OTA?

### Wireless Programming

Replaces physical USB cables entirely, enabling code deployment over Wi-Fi networks.

## 2. Practical Usage

### Sealed Enclosures

Update waterproof, ruggedized, or industrial hardware shells without breaking gaskets.

### Remote IoT Nodes

Maintain ceiling, roof, or field-mounted setups without physically extracting the board.

## 3. Library and Requirements

### Core Library

Built directly on the standard [ArduinoOTA.h](#) framework for seamless IDE network port mapping.

### Non-Blocking Loop

Requires a fluid loop architecture – traditional blocking *delay()* functions must be replaced with *millis()*.

# 5. More Sensors and Actuators

ESP32 can connect to many sensors and actuators.

- Temperature and Humidity Sensor:  
<https://randomnerdtutorials.com/esp32-dht11-dht22-temperature-humidity-sensor-arduino-ide/>
- MPU-6050 Accelerometer and Gyroscope:  
<https://randomnerdtutorials.com/esp32-mpu-6050-accelerometer-gyroscope-arduino/>
- OLED Display:  
<https://randomnerdtutorials.com/esp32-ssd1306-oled-display-arduino-ide/>
- Etc.



# Questions?